Guidodynamics Dynamometer

User Guide

Capstone Team F25-66

Modular Vehicle Dynamometer.

Authors: Liam Homburger, Aidan Bennett, Brennan Koontz, Sam Robie, Flavio Rios-Aldava, Nate Burns, Steven Tran, Trevor Hart.

Client: Dr. Polina Brodsky

Project Advisor: Dr. Brian Gockel

Technical Advisors: Prof. Adam Duran, Kaleb Binger, Isaac Wheelis, Matthew Ellis, William Schimberg.

Date: April 2026

Notice: This document is intended for users of the Guidodynamics Modular Vehicle Dynamometer. For developer and hardware documentation, refer to the DAQ Developer Guide and DAQ Module Datasheet.

Contents

1 Overview **3**
- 1.1 Vehicle Compatibility . . . 3
- 1.2 Occupational Safety . . . 3
- 1.3 Software . . . 3

2 User Safety and Operating Checklist **3**
- 2.1 General Operating Conditions . . . 3
- 2.2 ESTOP Test . . . 4
- 2.3 Pre-Operative Checklist . . . 4
- 2.4 Shut-Off Checklist . . . 5

3 Software Installation **6**
- 3.1 System Requirements . . . 6
- 3.2 Dependencies . . . 6
  - 3.2.1 Python . . . 6
  - 3.2.2 Github Desktop . . . 6
  - 3.2.3 Grafana . . . 6
- 3.3 Repository Setup . . . 7
  - 3.3.1 GitHub Desktop . . . 7
  - 3.3.2 Online Repository . . . 8
- 3.4 Network Settings . . . 9
- 3.5 runMe.bat . . . 12
- 3.6 Configuring InfluxDB . . . 12
- 3.7 Configuring Grafana . . . 14
  - 3.7.1 Adding InfluxDB Data Source . . . 14
  - 3.7.2 Loading Dashboards . . . 17
  - 3.7.3 Testing Grafana . . . 18

4 Software Use **21**
- 4.1 Starting the System: Local Mode . . . 21
- 4.2 Starting the System: Remote Mode . . . 21
- 4.3 Importing Grafana Dashboards . . . 21
- 4.4 Exporting Grafana Dashboards . . . 23
- 4.5 Generating New Dashboards . . . 24
- 4.6 Exporting Data to CSV . . . 26
- 4.7 View Previous Data with Grafana . . . 26
- 4.8 User Terminal . . . 27
  - 4.8.1 Setup . . . 27
  - 4.8.2 Gear Ratio Calculation . . . 27
  - 4.8.3 Manual Load Control . . . 27
  - 4.8.4 System Configuration . . . 28
  - 4.8.5 Run Plans . . . 28

5 Further Information **29**
- 5.1 Calibration . . . 29
  - 5.1.1 Overview . . . 29
  - 5.1.2 Torque Calibration . . . 29
  - 5.1.3 Moment of Inertia Calibration . . . 30

6 Guidance on Interpreting Data, Engine Types, and Configurations **30**
6.1 Data in Grafana . . . 30
6.1.1 Smoothing Factor . . . 31
6.1.2 SAE Correction Factor . . . 31
6.2 Engine Types . . . 31
6.2.1 Internal Combustion - Naturally Aspirated . . . 31
6.2.2 Internal Combustion - Forced Induction . . . 32
6.2.3 EV Powertrains . . . 32
6.3 Configurations . . . 33
6.3.1 Physical . . . 34
6.3.2 Controls Configurations (Run Plans) . . . 35
6.3.3 High Torque . . . 35
6.3.4 Smooth Start . . . 36

7 Appendix **37**

A Data Processing/Transformation Script **37**

B Moment of Inertia Script **38**

1 Overview

This document contains information relevant to an user of Capstone Team F25-66's Modular Vehicle Dynamometer. Any user seeking to go beyond use of the dynamometer should reference the DAQ Developer Guide, and the DAQ Module Datasheet, linked here.

1.1 Vehicle Compatibility

The dyno is compatible with the following vehicle specifications:

- **Power Output:** <= 100 hp
- **Track Length:** 56-75 in
- **Track Width:** 44-53 in
- **Tire Width:** <= 10 in
- **Top Speed:** 20-90 mph

1.2 Occupational Safety

The dynamometer presents several hazards that all users must understand before, during, and after operation. Anyone present in the same room as the dyno must take appropriate precautions, and the **User Safety** section of this document must be read and understood in full prior to operation.

Failure to comply may result in serious injury or death.

1.3 Software

The Dynamometer comes with comprehensive software to make data collection and visualization easy. The user software stack ingests data from the DAQ module, stores it in a database (InfluxDB), and displays it using Grafana. The software sections of this guide contain step by step instructions for how to configure this system on a Windows 11 computer.

At this time, the system does not virtualize. Keep in mind these steps will install the software on the top level on the computer, potentially interfering with other programs.

2 User Safety and Operating Checklist

The dynamometer is contained within a specialty built room that shields occupants of the adjacent classroom from most hazards, however, precautions must be taken.

2.1 General Operating Conditions

- The following PPE is required for dyno operation: **safety glasses, closed-toe shoes, long hair tied back, ear protection**
- A minimum of three people must be present during operation, with one stationed by the emergency stop button in the classroom and one stationed by the emergency stop button on the control box.
- The doors between the dynamometer room and the classroom must remain closed while in operation.
- If an ICE powertrain is being tested, the ventilation system must be enabled by turning the switch near the main IBC water tank, and the garage door opened. No spectators shall be allowed behind the dynamometer while in operation.

- For improved ventilation, it is recommended that flexible ducts be connected directly to the exhaust outlet(s). If the gas detection alarm activates during operation, stop the engine and move to fresh air.

- If users must be in the dynamometer room (e.g. in vehicle chassis configuration), they should be clear of the front and rear of the vehicle.

2.2 ESTOP Test

Additionally, before every session, the dynamometers safety system must be tested.

1. Ensure the main power connector is unplugged, and the master power switch it set to off. Unplug both pumps from their L5-20 connectors on the bottom of the panel.

2. Ensure all ESTOP buttons are in the released position. Plug in the mains power connector, and turn the master power switch to on.

3. Press the blue reset button, and press an ESTOP button. Observe the stack light turn red.

4. Repeat steps two and three for each of the two ESTOP buttons.

If the stack light does not turn red or does not light up at all, for any ESTOP, do not use the dynamometer and alert an TA or Professor.

After testing each ESTOP, set the master power switch to off, unplug the dynamometer from the mains, and plug in both pump's L5-20 connectors.

2.3 Pre-Operative Checklist

Complete the following checklist before any dynamometer session:

1. Complete the ESTOP Test as prescribed in 2.2

2. Ensure all conditions in 2.1 are met.

3. Set the **Main IBC Valve** (located at the bottom of the IBC tank) to the OPEN position.

4. Ensure that both ball valves (located at the outlet of the pump and the outlet of the return tank) are set to the OPEN position.

5. Inspect rotating machinery, including user DUTs (devices under test), and ensure fabric, rope, and other loose items are secured.

6. If in vehicle mode, ensure vehicle is properly strapped to the dynamometer and is unable to move. Also ensure that the water brake shaft pointing towards the left tire is covered to avoid tire damage.

    - Straps must connect to rigid frame members. **Do not connect straps to body work, suspension components, or drivetrain components**.

7. If in engine or brakes mode, ensure test stand is secured at all available points.

8. Turn the master power switch to the ON position and press the blue reset button. The stack light should turn green.

2.4 Shut-Off Checklist

Complete the following checklist after a dyno session:

1. Turn the master power switch to the OFF position.
2. Set the **Main IBC Valve** (located at the bottom of the IBC tank) to the CLOSED position.
3. Turn off the ventilation system.
4. Turn off lights and lock all doors to the dyno bay.

3 Software Installation

3.1 System Requirements

- Windows 11
- 8 GB RAM
- Ethernet Network Adapter [1]

3.2 Dependencies

3.2.1 Python

Python 3.13 or later must be installed. Lower versions of python 3 can be used, but are not guaranteed to work. To check this, open command prompt, and run the following:

```
python --version
```

You should see:

```
Python 3.x.x
```

If you do not see your python version, or you do not have Python 3.13+, install the latest version of python (here) on your computer before proceeding.

Long PATH Names

Depending on your computer, the python installer may ask to adjust path lengths, universal path settings, or install CPython. Respond "y" to these requests.

3.2.2 Github Desktop

Make sure Github desktop is installed on your computer.[2] Default settings are OK. Then sign in to your Github account.

Download the latest version here

3.2.3 Grafana

Download Grafana from this website. Make sure to download it here, instead of the Grafana Cloud website. Select version 12.3.2 Enterprise edition.

[1] Even if your computer does not have a physical Ethernet port, you likely still have an virtual Ethernet network adapter. You will still need an USB to Ethernet adapter dongle

[2] Install the Github desktop application even if you already have Git, unless you are sophisticated enough to set up Git LFS on your own

Keep all default settings while going through the Grafana setup steps. After the installation finishes, restart your computer. [3]

3.3 Repository Setup

The GitHub repository can be downloaded multiple different ways. For use when the program is still under development, it is recommended to use GitHub desktop to install and update the program on your local computer. Alternatively, the program can be downloaded as a .zip file, which is a quicker but harder to update method. See the appropriate subsection for how the program should be downloaded and installed on your personal machine.

Which Option

When in doubt, use the Github Desktop method 3.3.1 (SKIP 3.3.2). It is more stable and easier to update.

3.3.1 GitHub Desktop

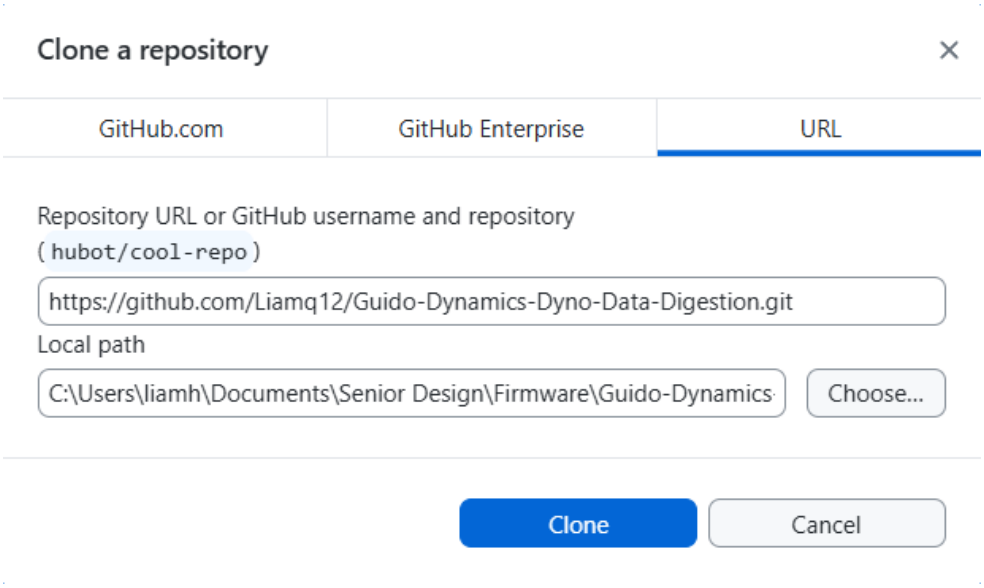
Open Github desktop and navigate:
File → Clone Repository Select the URL menu, and enter the following URL: [4]

https://github.com/Liamq12/Guido-Dynamics-Dyno-Data-Digestion.git

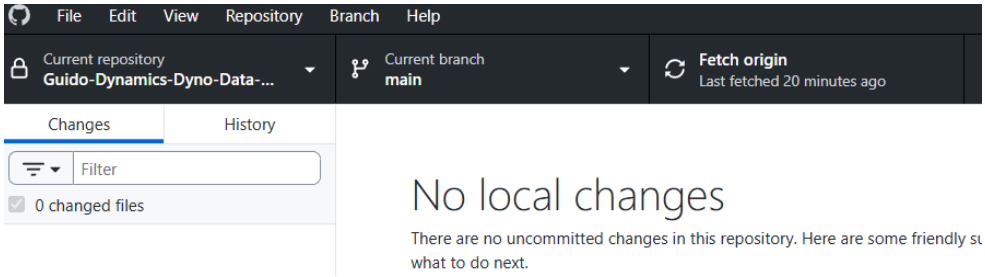
Select a path to your preferred location on your computer, and click "Clone".

[3] The default installation directory should be called "GrafanaLabs" in your Program Files directory

[4] If you are installing this after June 2026, the repository should be public, and should allow you to clone. If before June 2026, you may need to request access to the repository



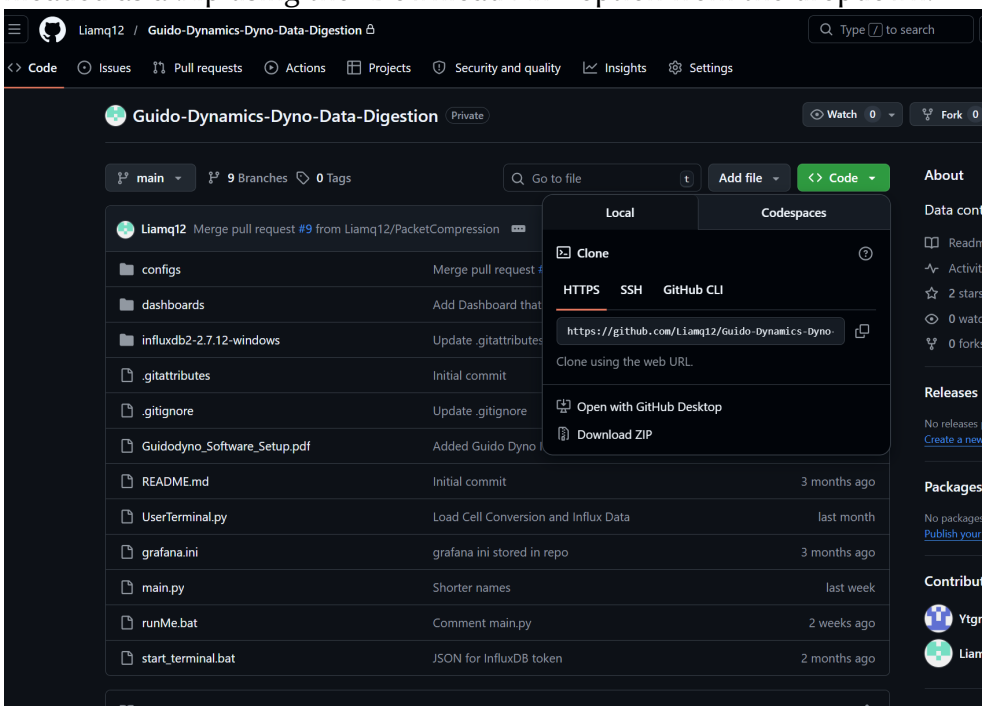
If prompted, click "Initialize Git LFS". Once the repository has cloned, click "Fetch Orgin" a the top of the Github desktop application. Make sure to fetch orgin periodically to get the latest software updates.



The Guidodyno software should now be installed in the directory you listed from the cloning step.

3.3.2 Online Repository

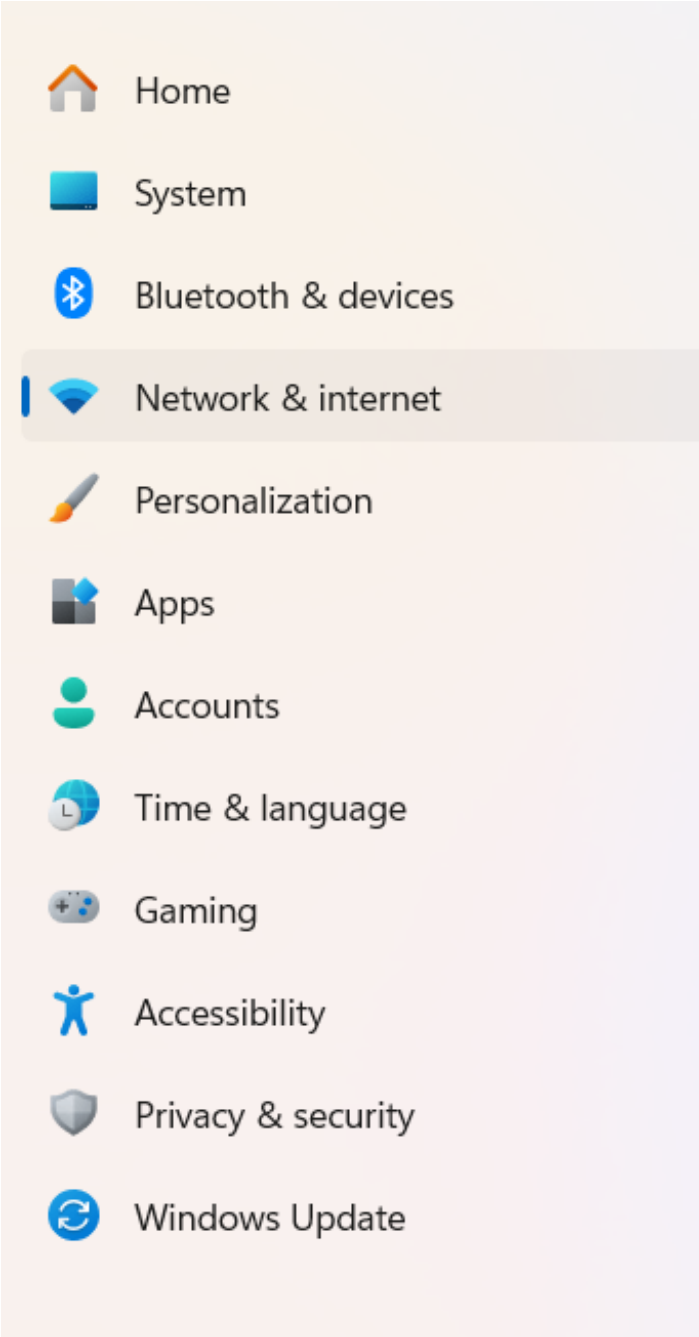
Navigate to 'https://github.com/Liamq12/Guido-Dynamics-Dyno-Data-Digestion'. From the green "Code" button, the user can see how the program can be downloaded using git if they are familiar. The program can also be downloaded as a .zip using the "Download ZIP" option from the dropdown.



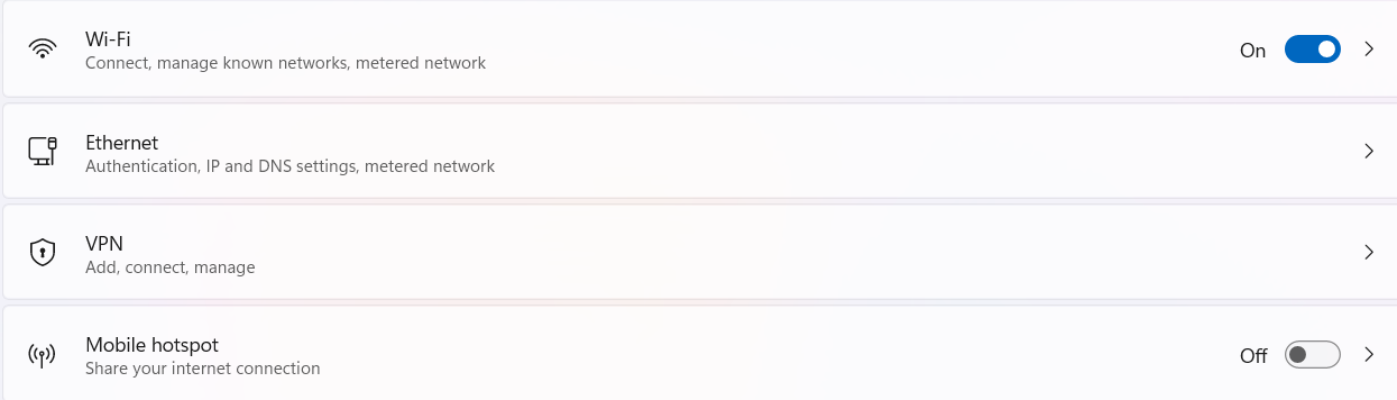
The program should be extracted, if applicable, and the folder location should be noted.

3.4 Network Settings

In the windows start menu, search for "Settings" and navigate to the "Network and Internet" tab.



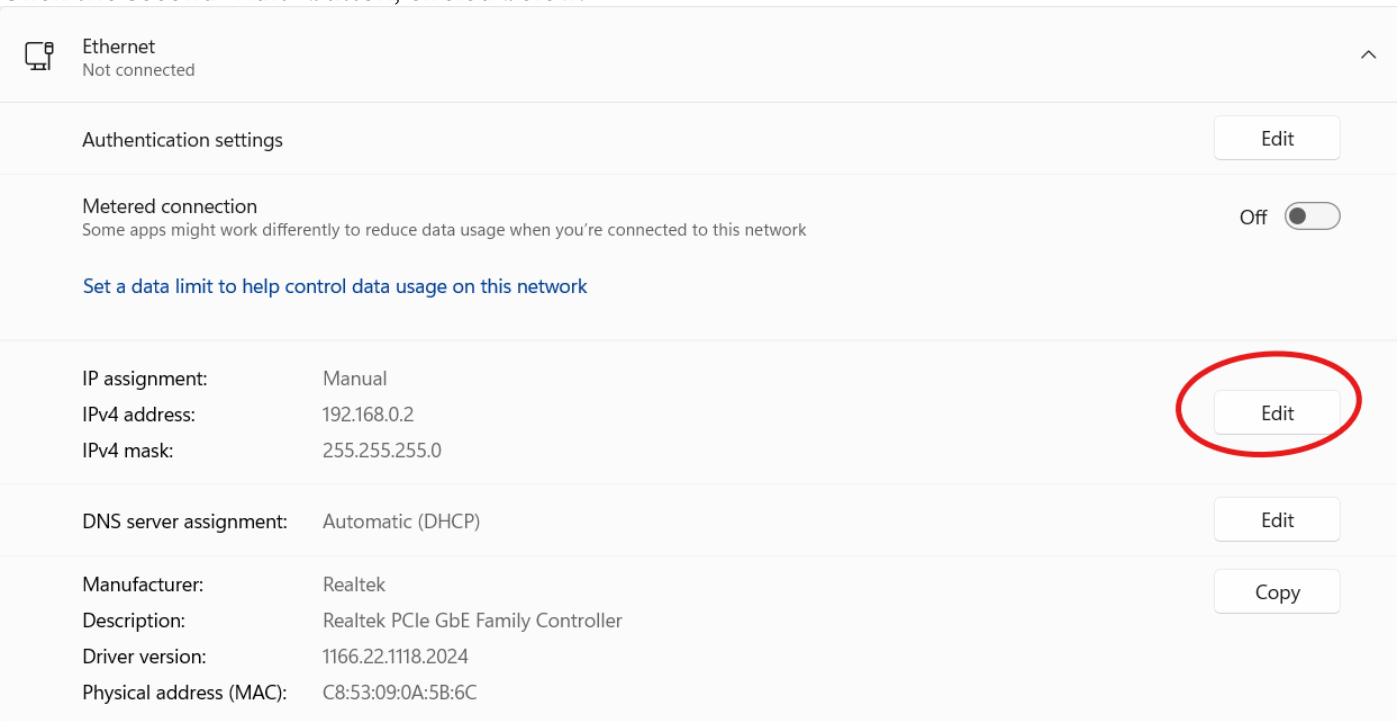
If you have a physical Ethernet port, click the menu button called "Ethernet"



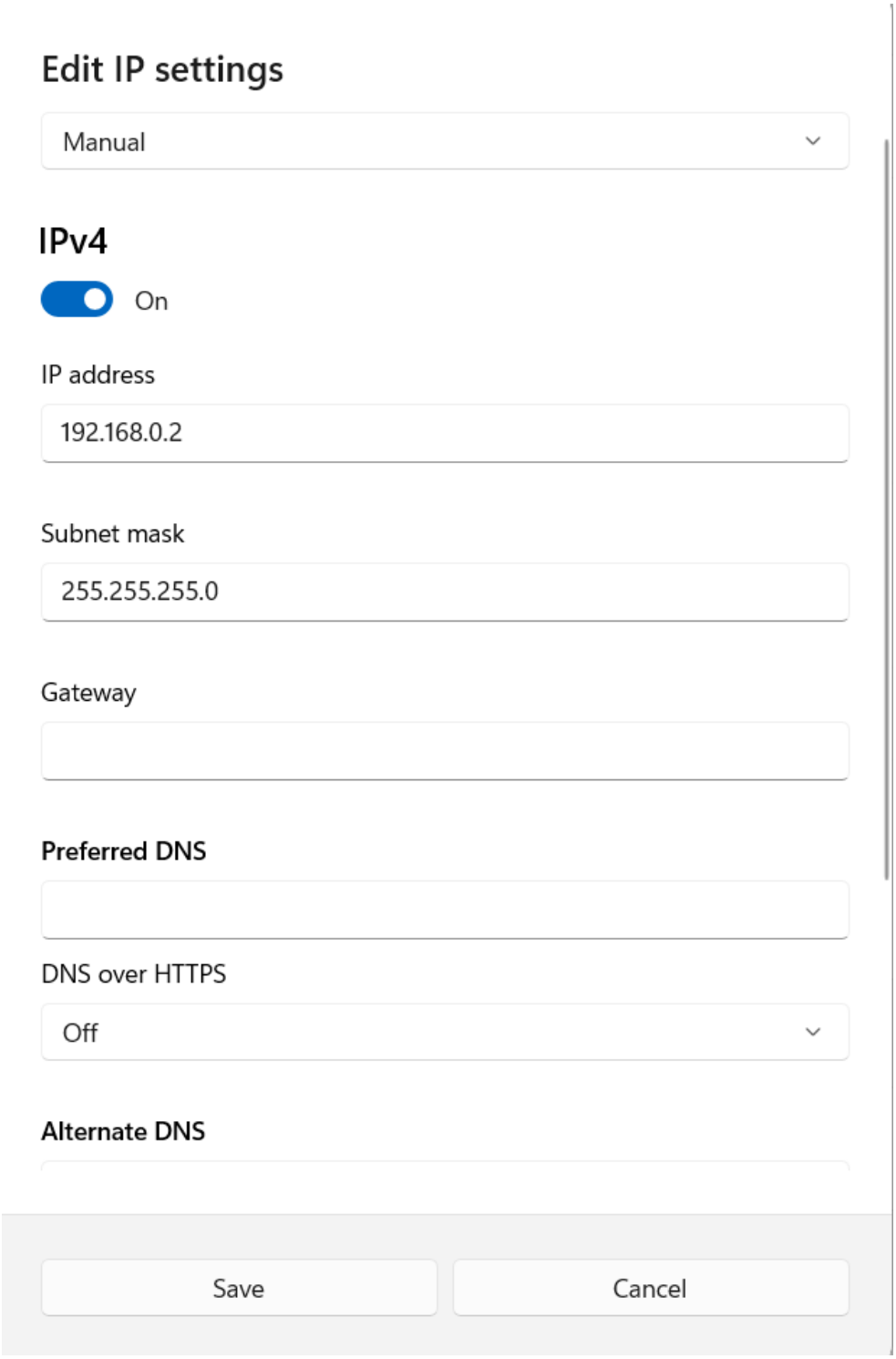
No Ethernet Menu

If you do not have an physical Ethernet port on your computer, you may need to navigate further menus to find this Ethernet network settings menu.
On Windows 11, the virtual Ethernet adapter can be found by clicking "Advanced Network Settings" at the bottom, and then clicking "Enable" on the Virtual Ethernet Adapter. IP address information is edited by clicking "View Additional Properties" within the adapter.
If you cannot find this menu, skip to step 3.5.

Click the second "Edit" button, circled below.



Under "Edit IP Settings", select manual.
Turn on IPv4.
For IP address, enter "192.168.0.2".
For subnet mask, enter "255.255.255.0".
Leave gateway field empty.
Leave perferred DNS field empty.
Under "DNS over HTTPS", select off.
Leave alternate DNS field empty.
Again, under "DNS over HTTPS", select off.
Turn off IPv6.
Click save.



3.5 runMe.bat

The runMe.bat file, located in the top level of your cloned repository, will start the necessary software to operate Guidodyno. Navigate to this file in file explorer, right click on the file, and run as administrator. [5] If a blue menu pops up, select "More Info", and then "Run Anyways".

This script starts the necessary software (InfluxDB, Grafana, UDPtoInflux Bridge "main.py", and the user terminal) and will spawn many windows. Leave this windows running while using the system.

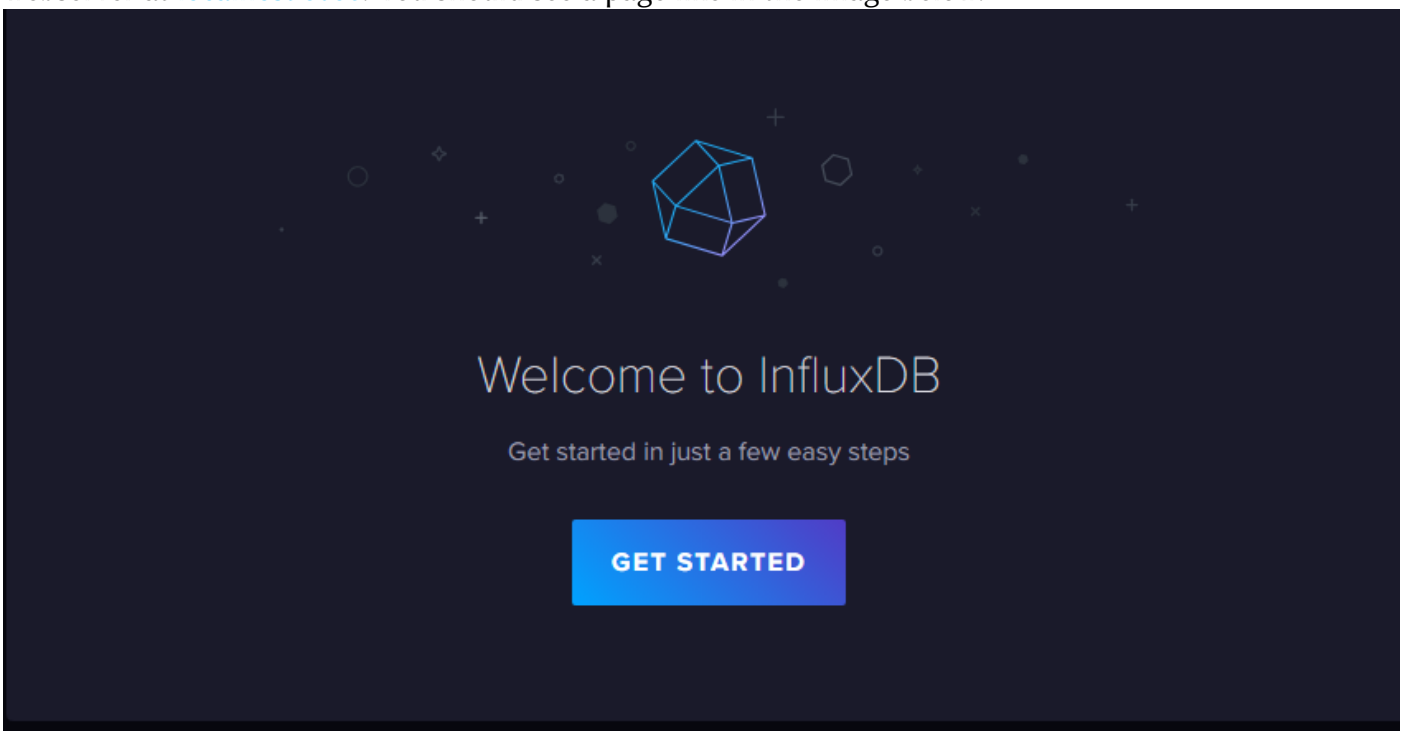
```
Starting UDP Ingest...
Starting User Terminal...
All processes started. Press any key to stop them...
Press any key to continue . . .
```

First Time runMe.bat

The first time running this script often requires "kicking" (press enter) the terminal. If the window is stuck at "The Grafana service was stopped successfully", press the enter key once to restart the program. Additionally, press "Allow" for any prompts that pop up.

3.6 Configuring InfluxDB

With the script running from the previous step, open a new browser tab and go to the InfluxDB local webserver at localhost:8086. You should see a page like in the image below.

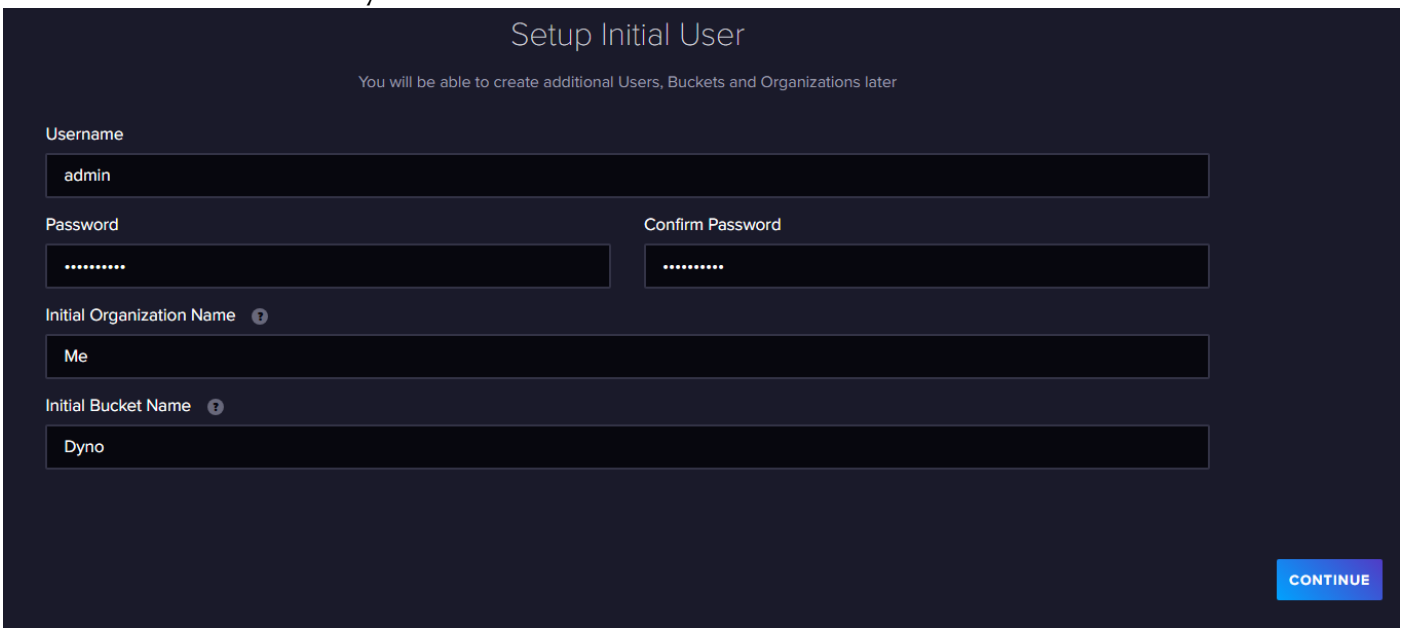


Error: Localhost Refused to Connect

Ensure the runMe.bat program is running (as administrator), and that you have fetched the latest version of the repository from Github Desktop.

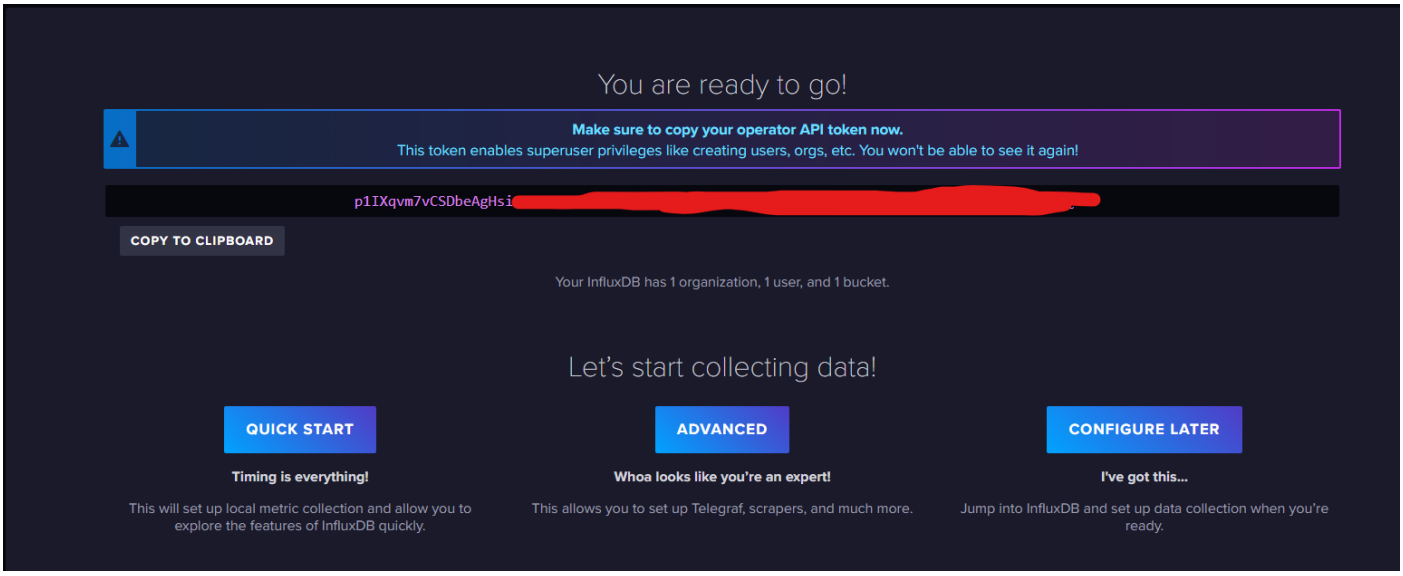
[5] This file is located with your other files cloned from GitHub. Unless changed in the clone step, the default location is: `C:\Users\[YOUR_USERNAME]\Documents\GitHub`

Click "Get Started" and make a username and password. [6] Set "Initial Organization Name" to "Me", and "Initial Bucket Name" to "Dyno". Click "Continue".



Since this information is stored locally, it is highly recommended to allow your browser to save your username and password.

You will now see your unique superuser token. This is only shown once, and you will not be able to view it again after moving from this page. Copy the token and save it somewhere you will remember, you will need it later!



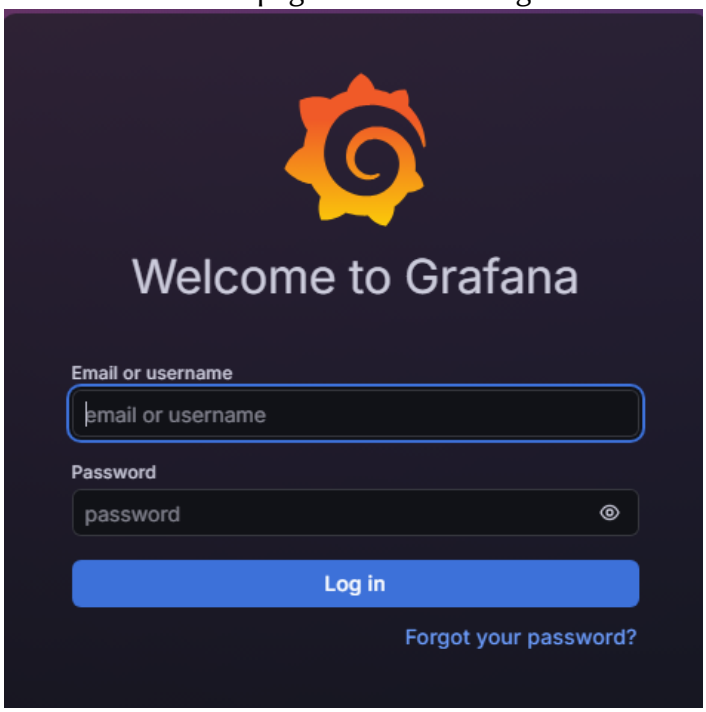
Click "Configure Later" once you have saved your token.

You are now done setting up InfluxDB. Feel free to close this tab.

[6] User data is stored locally, and not in the cloud or the Guidodyno Data Digestion repository. There is no way to recover a lost username/password, so make it simple. It is only accessible when signed into your Windows user account.

3.7 Configuring Grafana

With the runMe.bat script running from step 3.2.1, open a new browser tab and go to the Grafana local webserver at localhost:3000. You should see a page like in the image below.



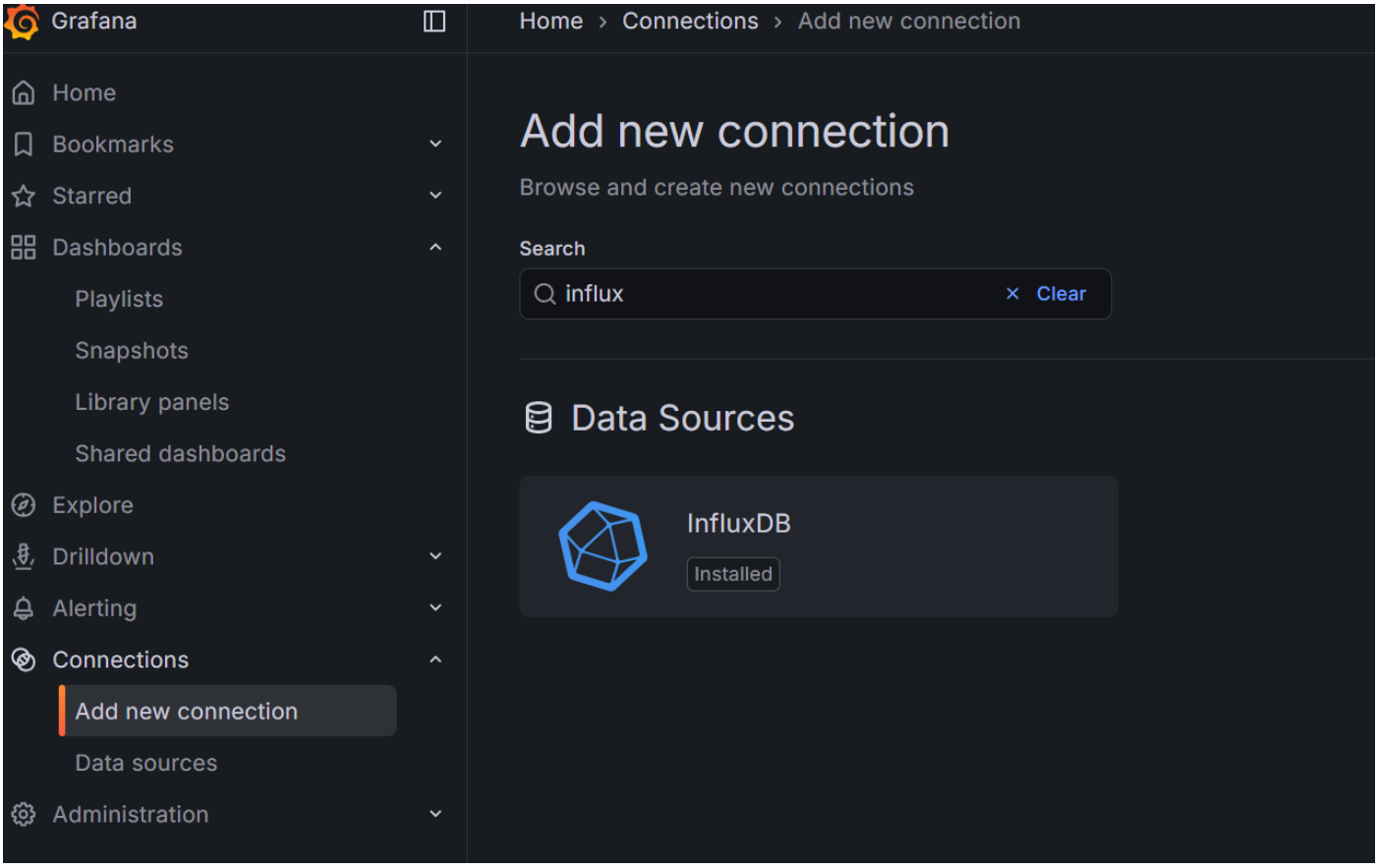
Error: Localhost Refused to Connect

Ensure the runMe.bat program is running (as administrator), and that you have fetched the latest version of the repository (With Git LFS enabled) from Github Desktop.

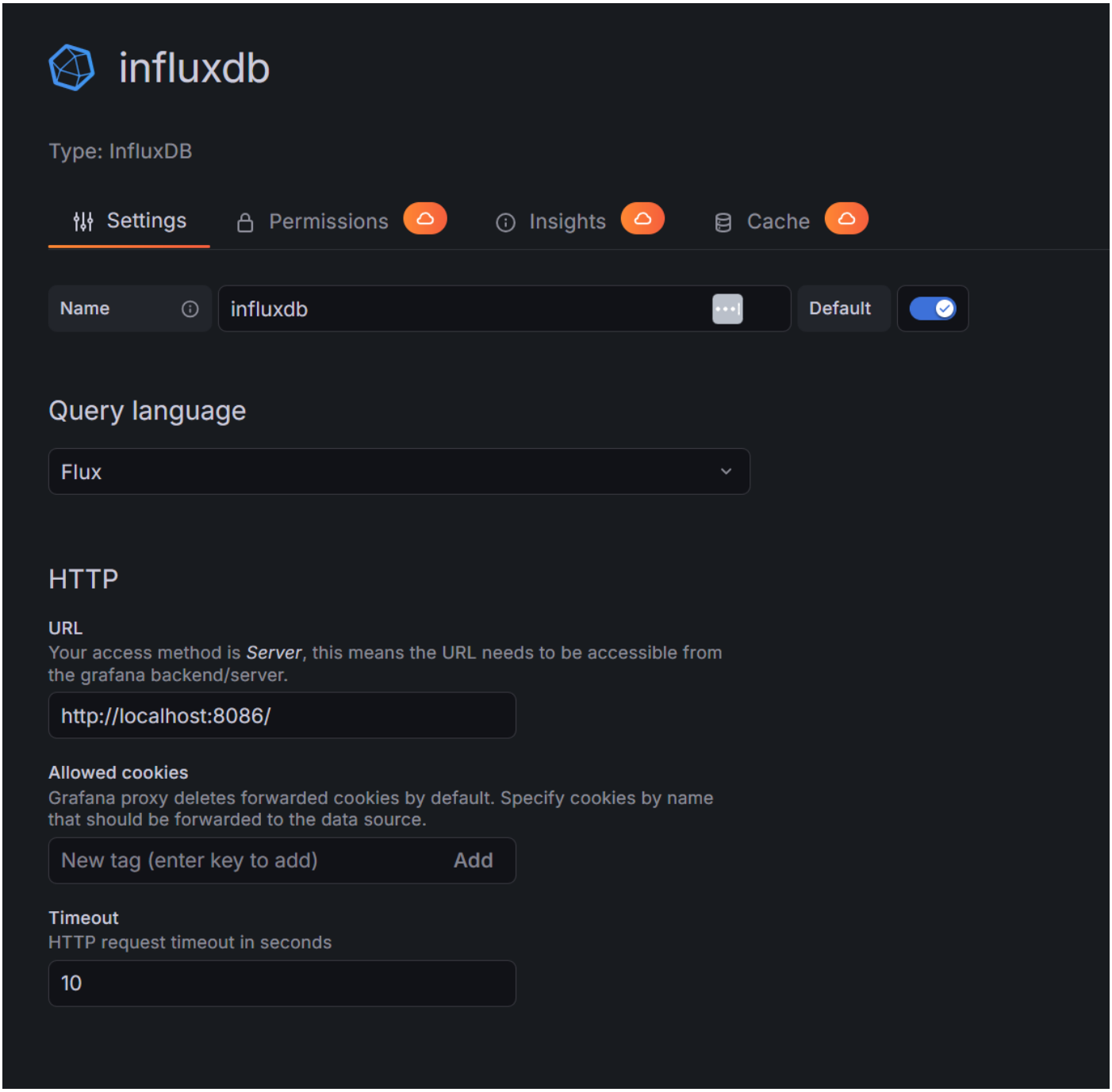
The default username is "admin", and the default password is "admin". It will then prompt you to change the password. Like InfluxDB, this is only accessible within your Windows user account, so retype "admin" as your new password.

3.7.1 Adding InfluxDB Data Source

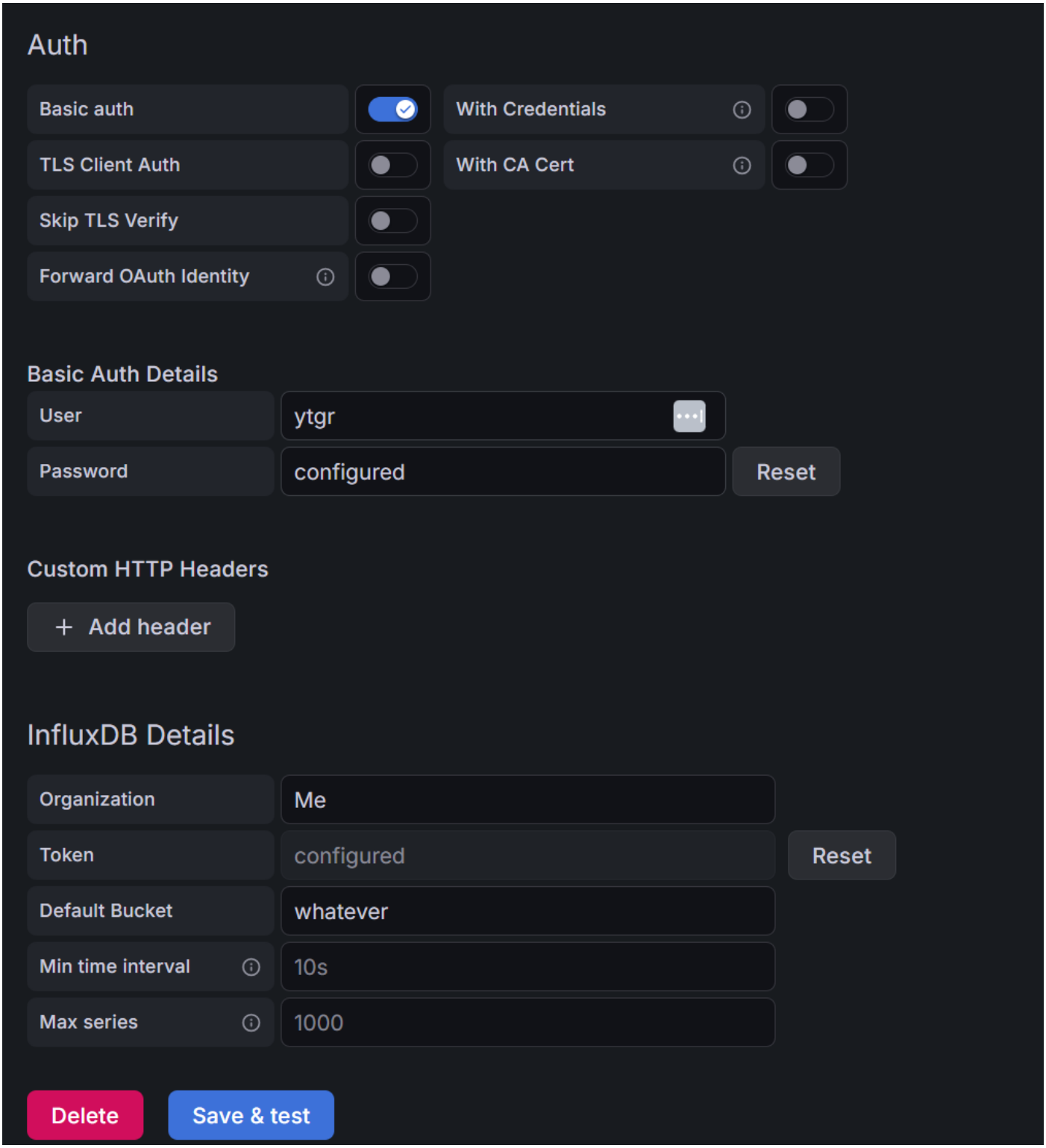
In Grafana, on the left side menu, navigate to Connections->Add new connection. In the search by, type in "influx". Choose InfluxDB, and click "Add new data source".



Configure the InfluxDB source as below, with the Query Language being Flux, and the URL being "http://localhost:8086/. Ensure the default setting is enabled.



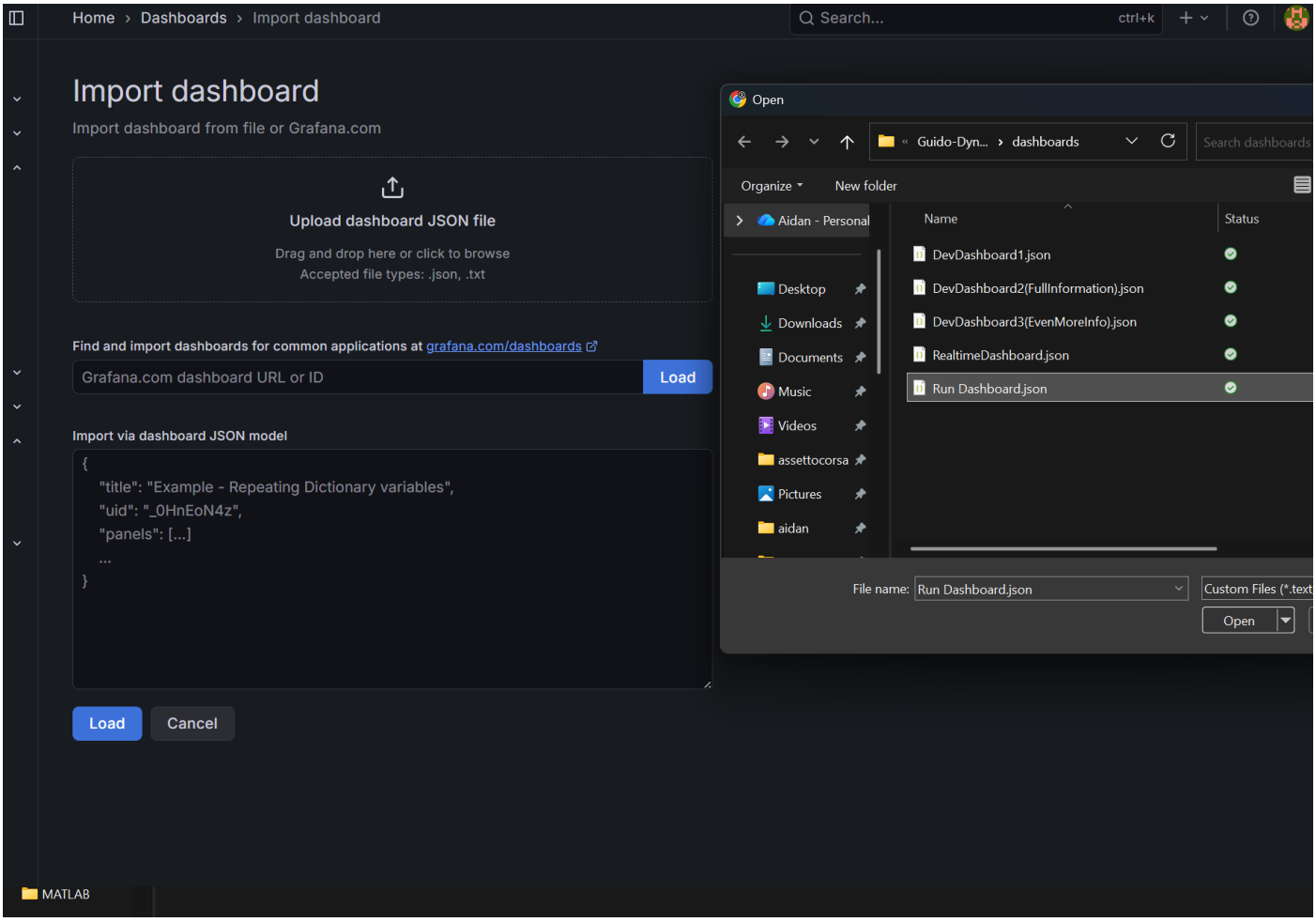
The bottom half of the InlfuxDB setup is shown below. Ensure the Basic Auth switch is selected. Enter your InfluxDB username and password in the appropriate fields. Enter the Organization, Token, and Bucket in the appropriate fields as well. Then select save and Test. Grafana should now be setup to pull data from InfluxDB.



3.7.2 Loading Dashboards

Within the downloaded program folder, there is a "dashboards" folder. Here lie the "RunDashboard.json" and "RealtimeDashboard.json", which can be imported into grafana. These are already setup for you, where the run dashboard groups data from individual pulls, and the realtime dashboard provides realtime data for debugging and other use cases.

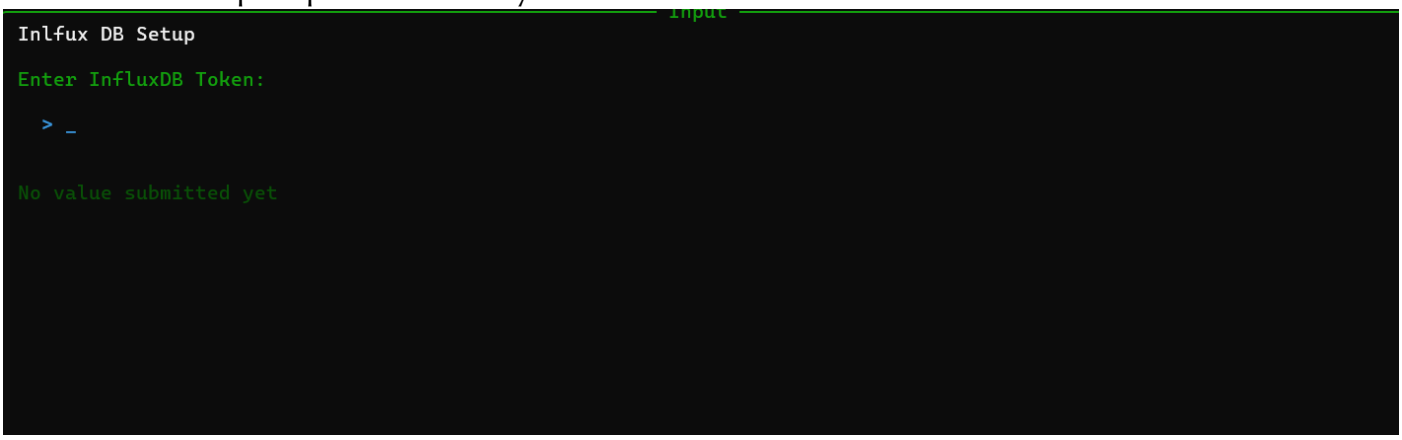
In the Dashboards tab of Grafana, select New->Import. From here, import the dashboards you want. For this tutorial, load the Realtime dashboard.



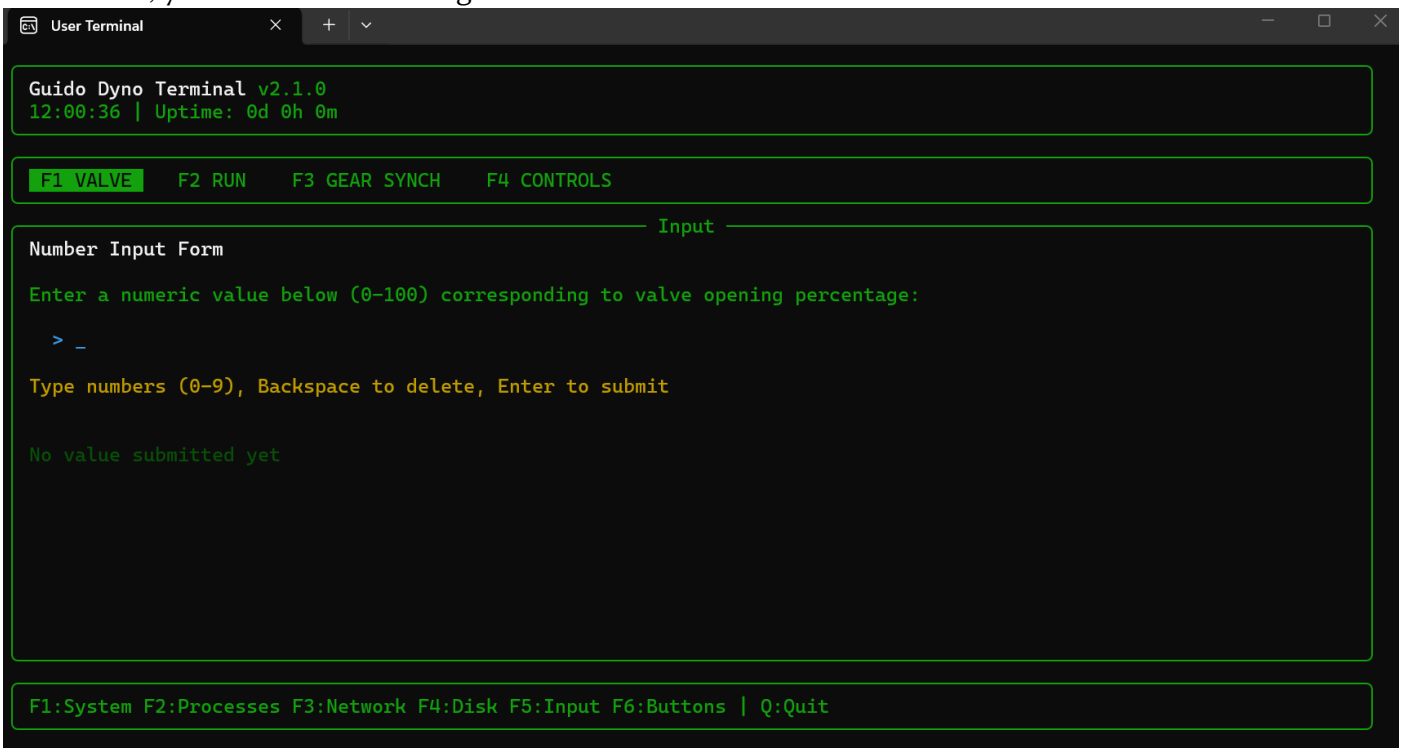
Within these dashboards, there is a textbox at the top titled "Bucket Name". Enter the bucket name you set up in the influxdb setup so that the dashboards can properly pull from your data.

3.7.3 Testing Grafana

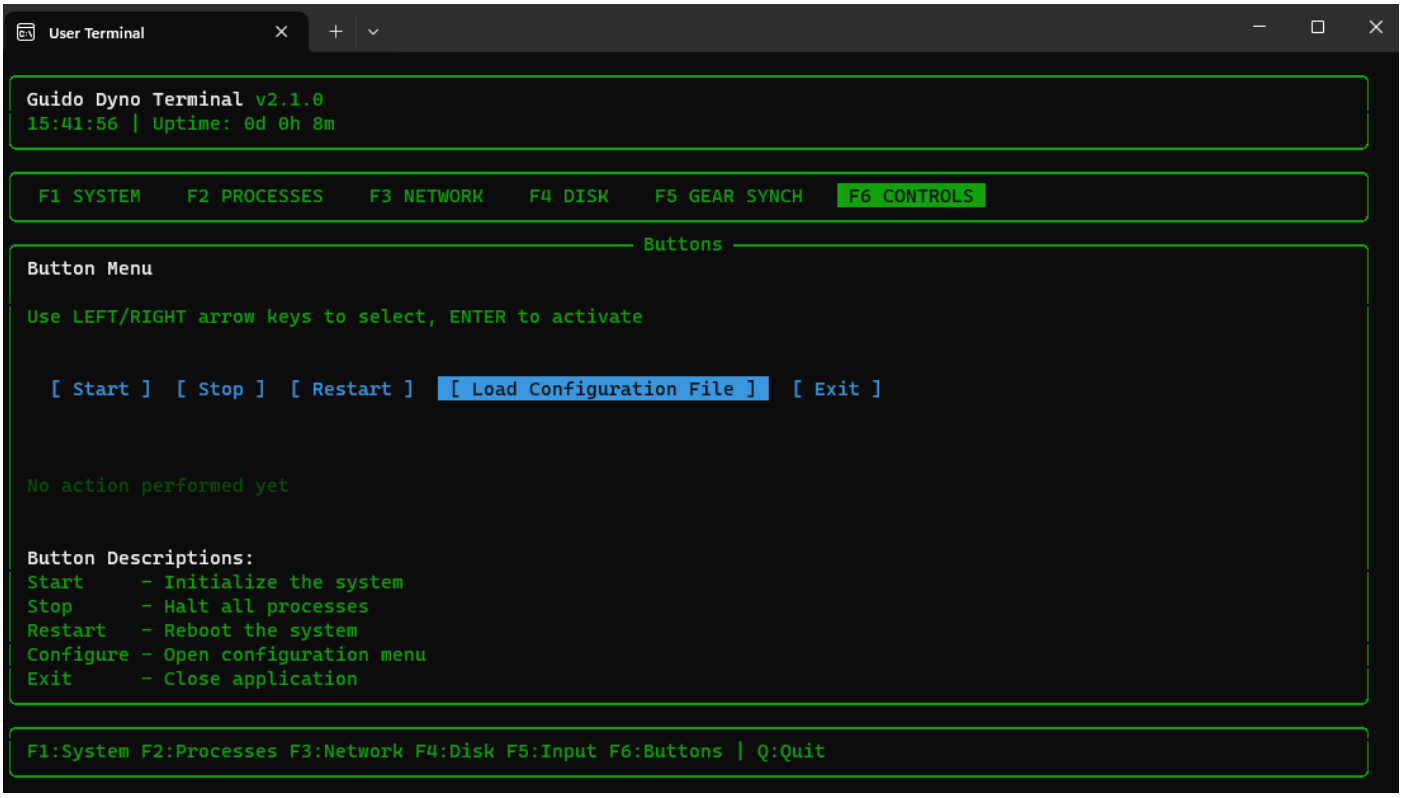
Run the runMe.bat program in the software folder as administrator, and look for the user terminal running in a CMD window. The splash screen looks like the image below. Enter your InfluxDB token, organization, and bucket when prompted. This is only done once.



Afterwards, you should see an image like below:



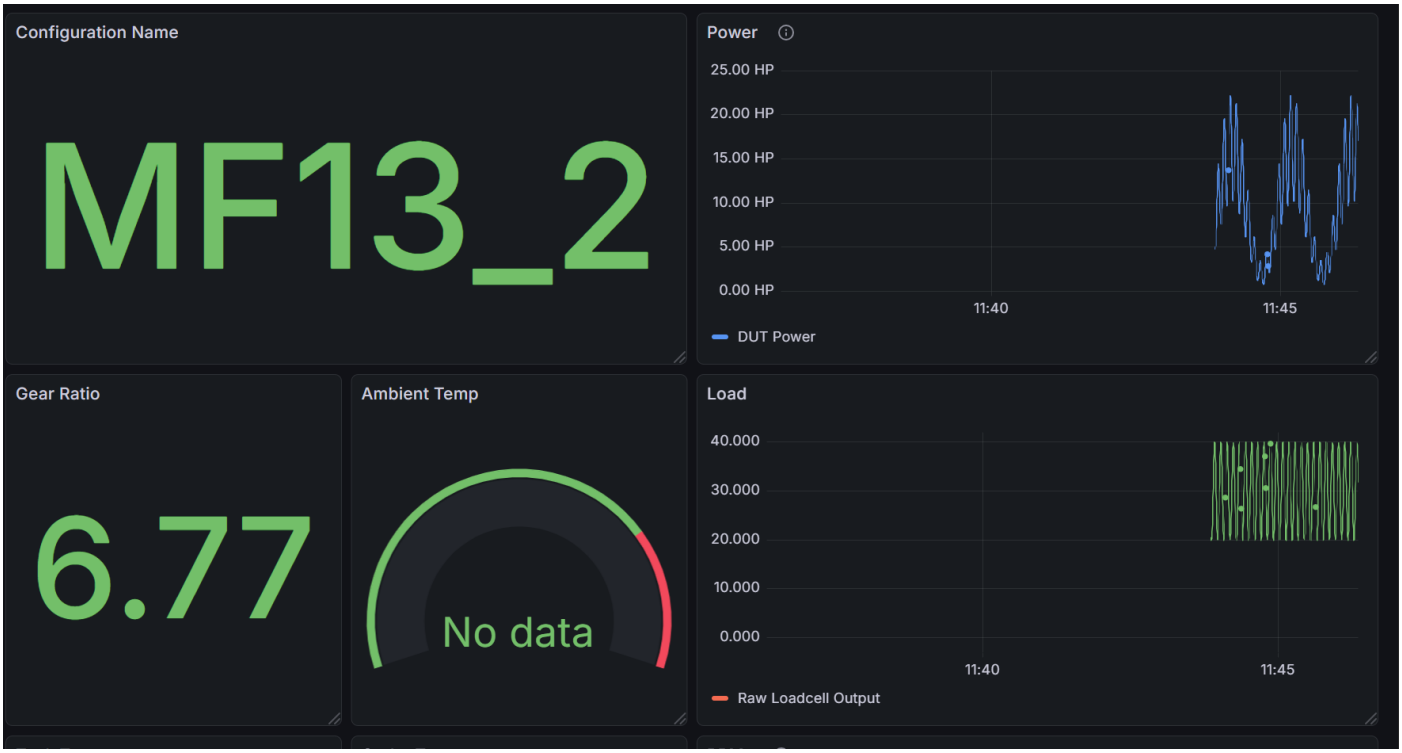
Click F6 to move to the Controls tab, then use the arrow keys to move to "Load Configuration File". Press enter to select.



Navigate to the config folder in the repository folder and open "VehicleConfiguration.json". This config folder is where Dyno configuration files are stored.

Back on the Grafana dashboard, click refresh in the top right[7]. Your stat visualization should now display some data! Once a configuration file is loaded, the configuration name should be uploaded. If the computer is not connected via Ethernet to the dyno, fake data will populate within the dashboard to ensure all the internal systems are working:

[7]The refresh button in the Grafana window, not the browser refresh button



If that worked, your system is configured and ready to collect data from the Guidodyno!

4 Software Use

4.1 Starting the System: Local Mode

The runMe.bat script, located in the top level repository folder, starts all necessary software to run the Guidodyno. Make sure the dyno is connected to your computer over Ethernet if you are collecting data before running this script. Always run the script as an administrator.

Ensure the pre-operative checklist in section 2.4 has been executed before operating the dynamometer.

That's it! Go to localhost:3000 to view live data. Go to localhost:8086 to download and view the data.

4.2 Starting the System: Remote Mode

The Mines Formula SAE Team hosts an InfluxDB and Grafana server available to all Dyno users. To access this system, go to fsaelinux:3000. This website is just for data visualization, not control.

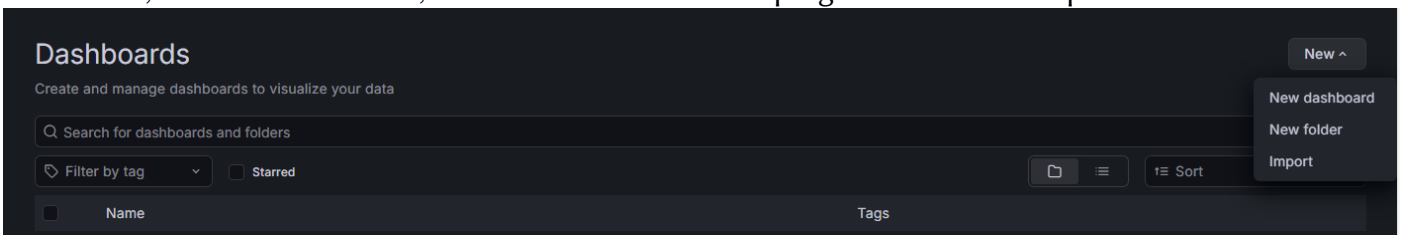
To operate the Dyno, navigate to your cloned github software folder, and run the script ./start_terminal.bat. This script will open a window where you can load configuration files, run plans, and start pulls. More information in section 4.8.

Global Protect

Due to Mines ITS policy, your computer must have the Mines Global Protect VPN installed and activated prior to using this website. If you do not have this installed, install it here: globalprotect

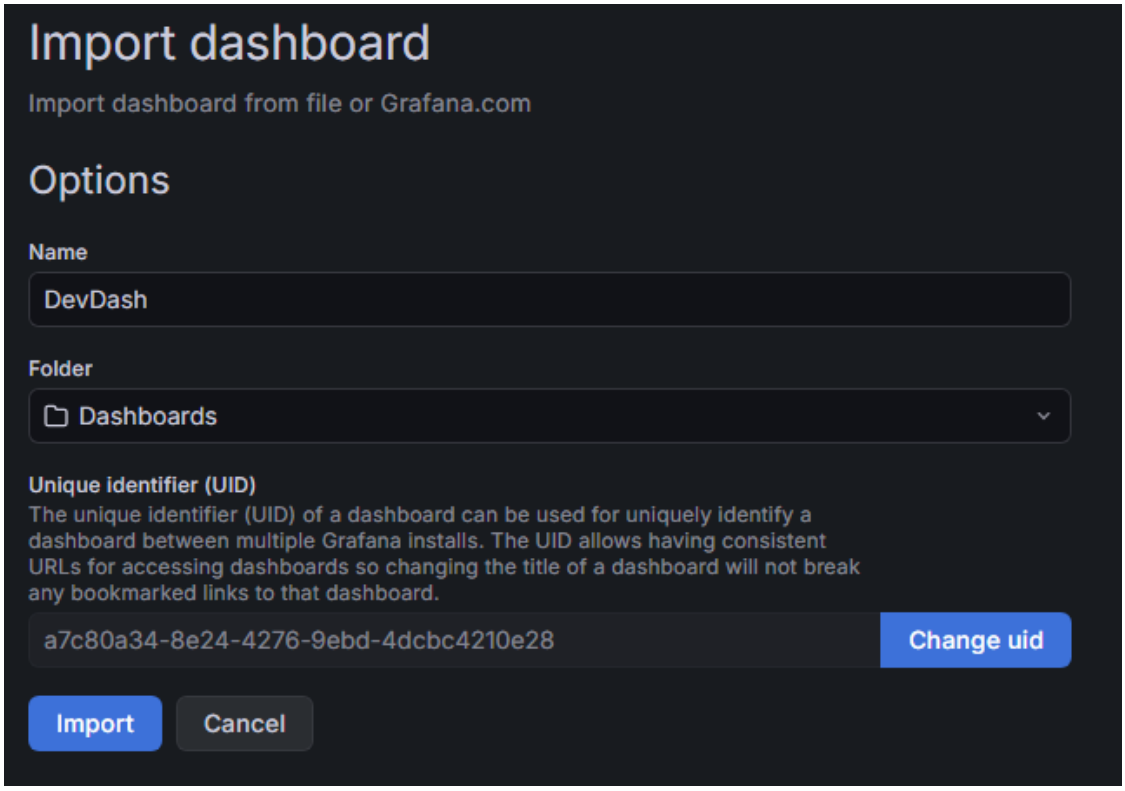
4.3 Importing Grafana Dashboards

In Grafana, select "Dashboards", and select "New" in the top right. Then click "Import".



Click "Upload dashboard JSON file", and navigate to the Dyno software repository. There is a top level folder called "dashboards" that contains dashboards made by other Dyno users you can use. "RealtimeDashboard.json" is one that is used for testing the system.

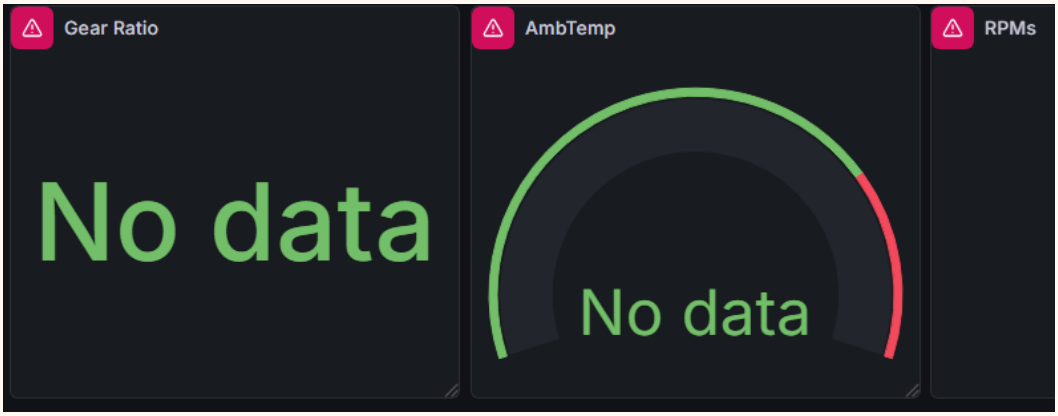
Select the dashboard you want to import, and give it a name. Leave the Folder and UID fields alone unless you need to change them.



Click "Import". The dashboard will now load.

Make sure to update the bucket name variable at the top with the bucket name that you used in InfluxDB setup.

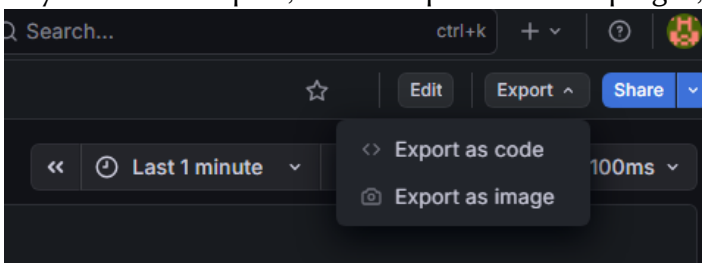
Bucket Name Mismatch



Some Dyno users use different bucket names for various reasons. A likely cause of visualization errors (red triangle in top left of visuals) is the wrong bucket name. When using either the Run Dashboard or Realtime Dashboard, the bucket name can be changed using the global variable at the top. When using other dashboards, click the three dots in the top right of a visualization, click "Edit", then change the bucket field to your bucket name. This needs to be done for each visualization, but only once after importing a dashboard. It is now saved on your computer, and can be exported with your bucket names.

4.4 Exporting Grafana Dashboards

With the Grafana dashboard you want to export, select "Export" in the top right, and click "Export as code".



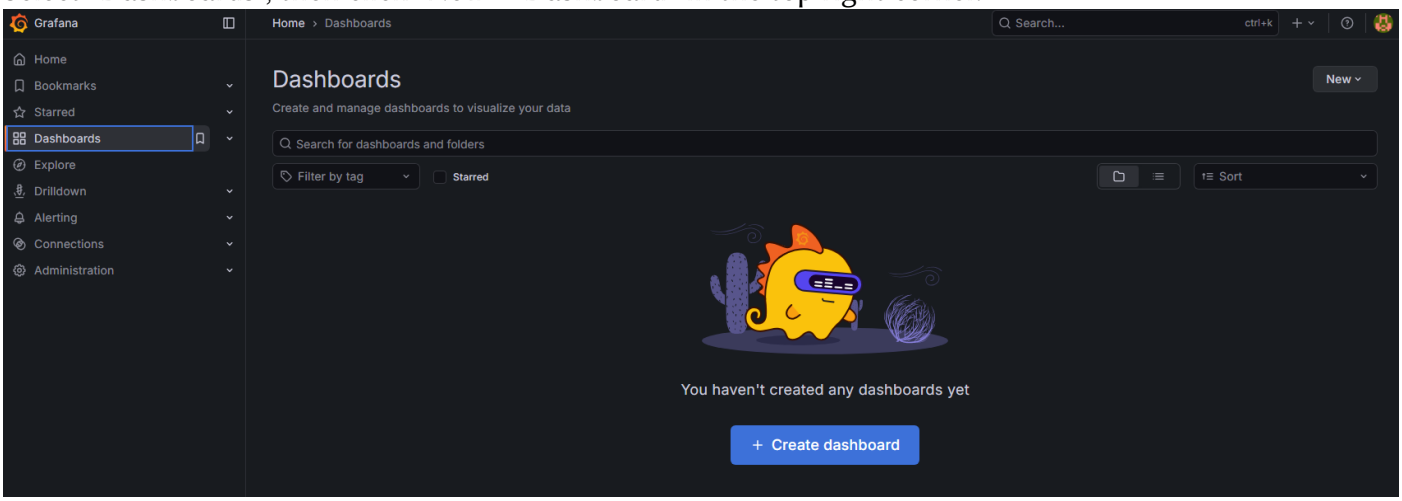
Click "Download file" in the bottom left of the download menu.

There is an shared dashboards folder in the software repository you can add your dashboard to if you would like to share with others.[8]

[8] Folder is named "dashboards"

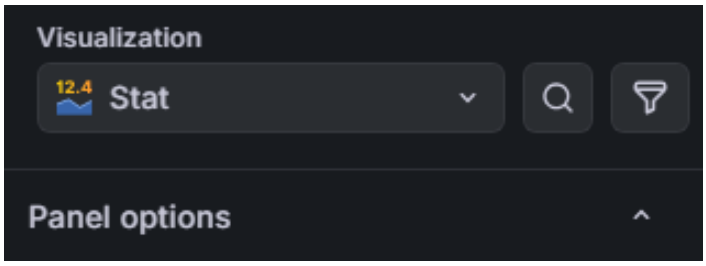
4.5 Generating New Dashboards

Select "Dashboards", then click "New">"Dashboard" in the top right corner.



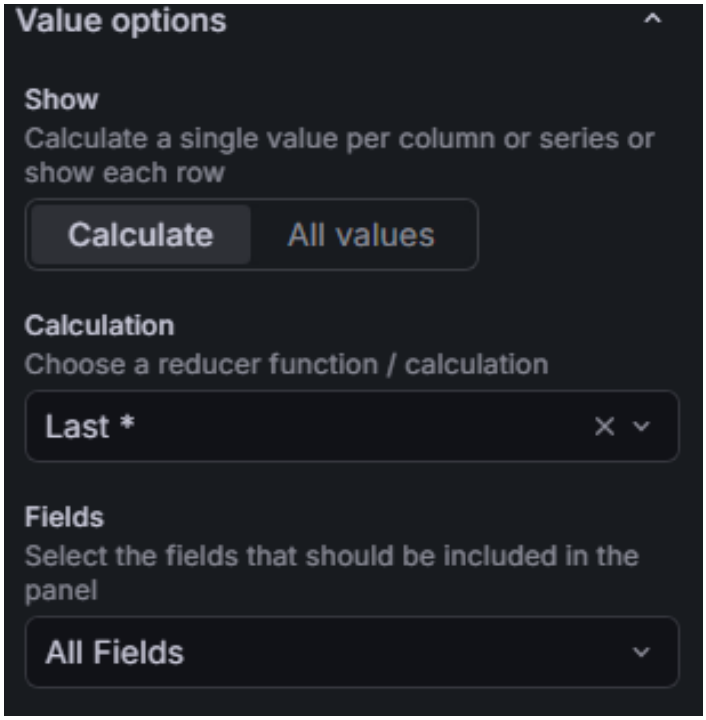
Click "Add visualization", then click on InfluxDB [9]

Select "Dashboards" from the left menu, click "Create dashboard", click "Add visualization", then click "influxdb" from the "Select data source" menu. This will open a new visualization pulling data from Influx.

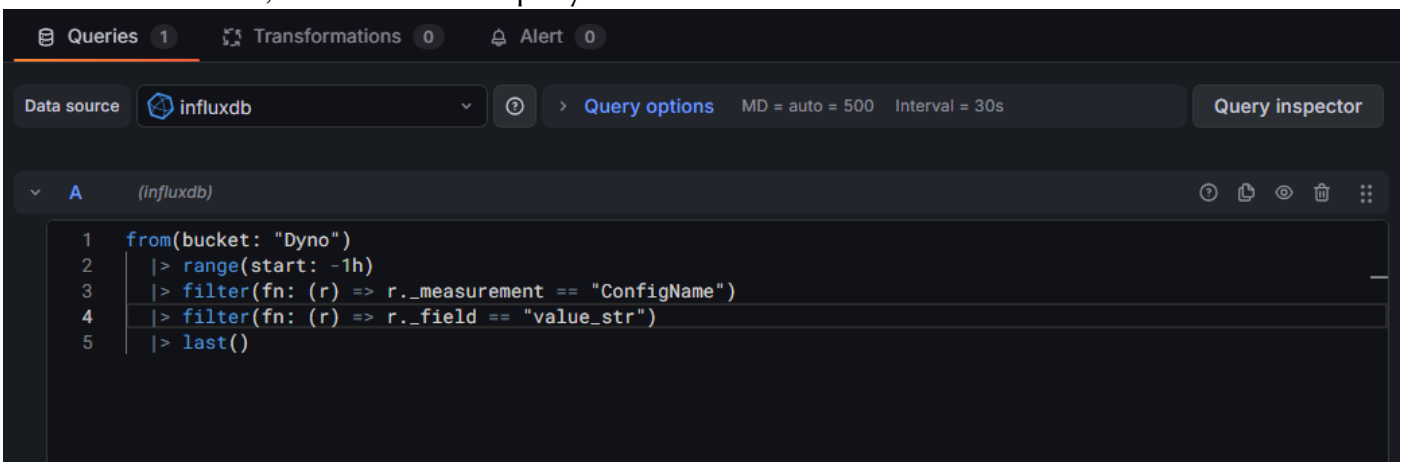


[9] Imported from step 3.7.1.

Also in the right side menu, under "Fields", select "All Fields"



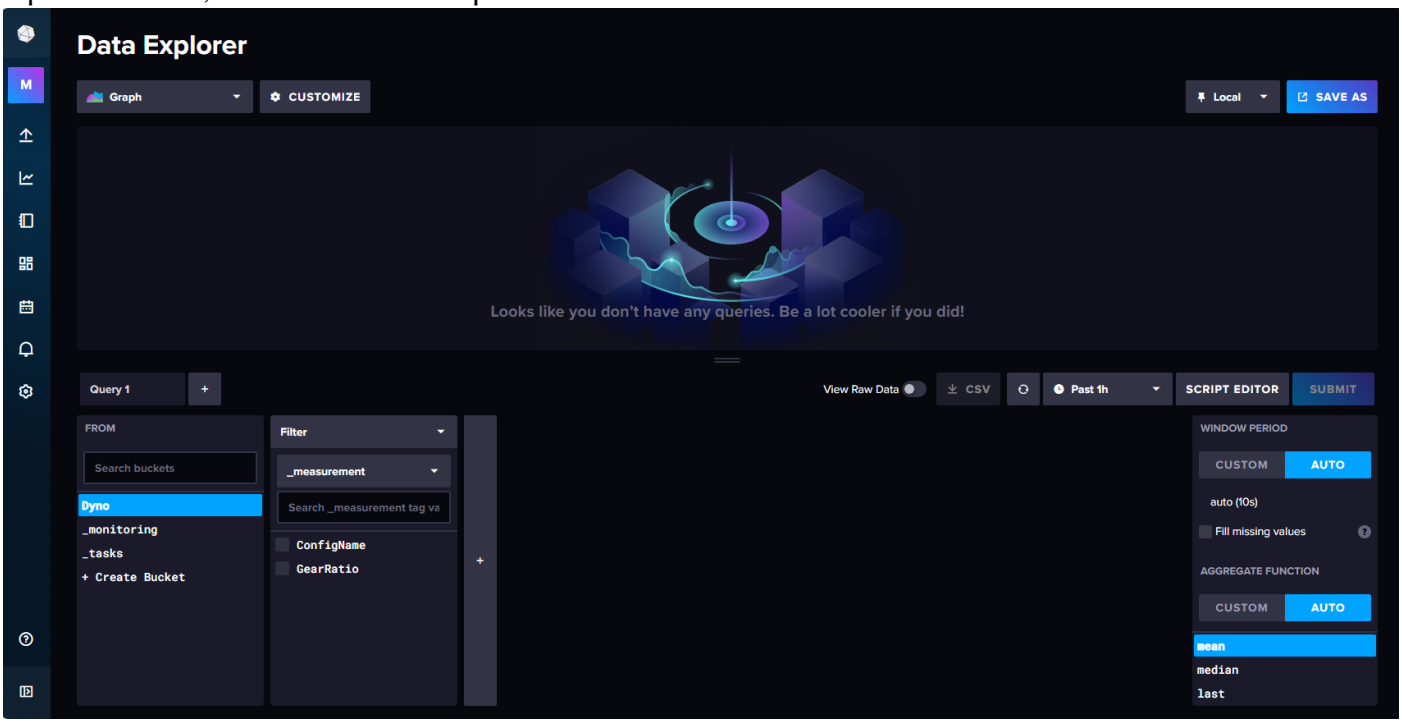
In the main window, an InfluxDB flux query can be written to retrieve data from InfluxDB.



(More info on this TODO)

4.6 Exporting Data to CSV

Open InfluxDB, and select "Data Explorer" form the left side menu.



Using the check boxes, select the data you want to export. Once selected, select how far you want to look back, set your window period (data rate), and click the blue "Submit" button. Once complete, click the CSV download button, next to the "View Raw Data" slider. This will download your selected data with UTC time basis information.

4.7 View Previous Data with Grafana

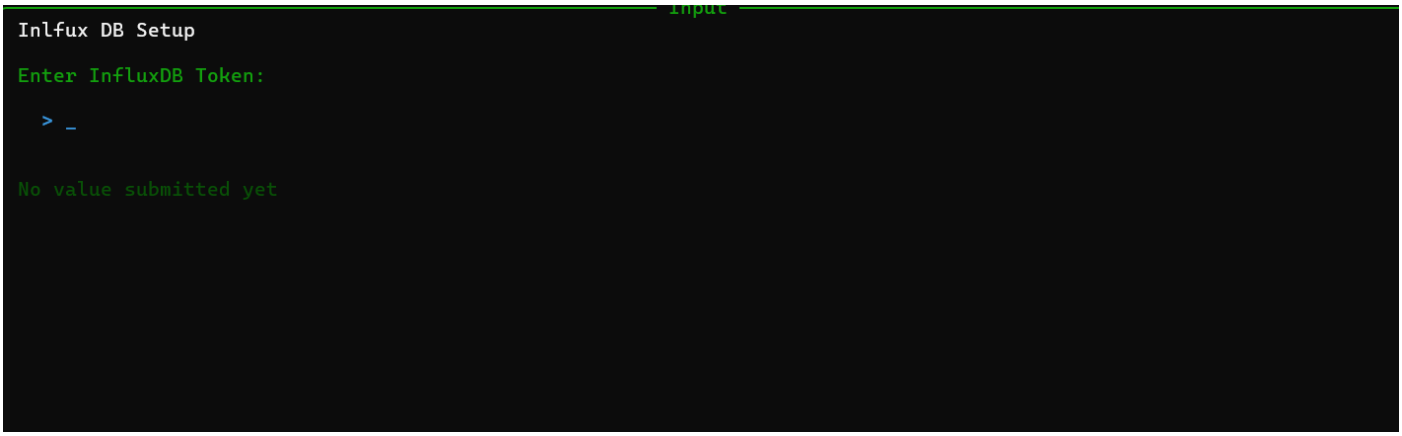
From Grafana, open the "Run Dashboard" (This will already be imported if using fsaelinux:3000). Widen the time scale using the time selector in the top right to the time period where the run/pull was performed. Open the "Select Run" drop down at the top of this dashboard to select your run, and generate the RPM vs power/torque curves.

4.8 User Terminal

The user terminal application allows engineers to manually control the dyno, and load automated ramp sequences.

4.8.1 Setup

The first time the user terminal is opened, the user must paste their InfluxDB token, bucket name, and org name into the terminal. This is a one time setup step.



Paste your InfluxDB token, bucket name, and org name from step 2.6.
This data can be changed anytime by going to F4>Setup InfluxDB, or by editing the config file configs>system>influxdb.txt.

The user terminal must be programmed with the stepper controller's pulses per revolution setting, as set with the DIP switches on the controller. This is set to 6400 by default, and the user terminal initializes with this value. **Only change this value F4>[Setup Valve] if it is changed on the stepper controller.**

Stepper Controller

The stepper controller is located inside the central control panel. Service must be performed only by qualified personnel with demonstrated competence in working with mains electrical systems.

4.8.2 Gear Ratio Calculation

Load the "GearRatioSyncConfig.json" file provided. With a system on the dyno (engine or vehicle), hold the throttle down and wait for the RPM to stabilize. Enter this RPM into page F2 while the RPM is held. Record the calculated gear ratio. In the vehicle configuration file, enter this calculated gear ratio in the "Gear Ratio" field of the JSON.

JSON Formatting

The gear ratio must be inputted as a numerical value, ie. there must not be quotations around the number in the JSON file.

4.8.3 Manual Load Control

The user terminal can manually control the dyno load by changing the position of the valve. To do this, go to window F1, and type a value 0%-100%. This will set the valve to this position until commanded otherwise.[10]

[10] This will not work if the dyno is in PID mode or ramp mode.

4.8.4 System Configuration

To load a system configuration file, first go to the configs folder, and create a vehicle file. VehicleConfiguration.json is provided as a template you can use to start.[11]

Name this file, and change the value for the field "Gear Ratio". See section 3.6.2 on how to calculate the gear ratio if it is unknown.

Gear Ratios

The gear ratio is affected by any of the applicable factors: gear selection, final drive, tire size, roller size, sprocket size, and more. As such, the user cannot simply "plug in" the selected gear ratio from a spec sheet, and it is recommended to undergo the gear ratio calibration outlined in the above section.

If your system has multiple gears, create a file for each gear with the relevant gear ratio.

The "Name" and "Version" fields are intended for the user to be able to differentiate between runs. It is recommended to keep the "Name" field abbreviated for readability and include the gear ratio. For example, the name "MF13_G3" may denote the Mines Formula 13th chassis in 3rd gear.

The "ID" field is used by the database, and must be unique for each vehicle and configuration, although these can be arbitrary.

Enter the desired run plan in the bottom section of the vehicle configuration. See the "HoldEx" and "RampEx" files in the Run Plans folder. The information from these files can be copied and pasted into the vehicle configuration. Alternatively, the run plan section of the vehicle configuration file can be deleted, and the user will need to load run plans separately. Be cautious of commas in the JSON file.

Before operation, go to F4>Load Configuration File and select your configuration file.

4.8.5 Run Plans

To load a run plan, first create an run plan configuration file. The files RampEx.json and HoldEx.json, located in the Run Plans folder, contain sample run plan configurations for a ramp and RPM hold, respectively.

These run plans can be loaded in the F4>Load Run Plan menu. Run plans loaded in this manner take precedent over any run plan integrated in a configuration file, if applicable. If the configuration file does not contain a run plan, it must be loaded manually as described in this section.

RPM Values and Gear Ratio

Run plans, integrated into the configuration file or loaded independently, use the engine RPM for control. This means the loaded gear ratio must be correct for the engine RPM desired.

Runs are started from the F2 menu, by pressing Enter.

See section 6.3 for more in-depth discussion on run plans and vehicle configurations.

[11] Only use the top section of this config file. The bottom is for RPM ramp settings.

5 Further Information

5.1 Calibration

5.1.1 Overview

The water-brake dyno style excels in steady-state (and near steady-state) conditions (constant or slowly ramping engine speeds). The moment of inertia is included in engine torque measurements, meaning the acceleration (constant and oscillatory) is calibrated out, providing much higher data accuracy. However, it must be noted that the dynamometer is not designed as an inertial dyno, and as such inertial calibration should not be held to the same standard as steady-state readings. As such, slow and stable ramps should be prioritized by users.

The rolling resistance of the dyno system has also been calculated, and is also included in engine torque measurements. This is the rolling resistance of the issolated dyno system - bearings supporting roller shafts, the power delivery belt, and other torque loss locations. The dyno does not calibrate out the rolling resistance of tires on the rollers, drivetrains, or chain drives for the engine stand configuration. This provides the most accurate "at the wheels" power measurement.

The calibration factors are exposed in the dyno_mechanical.json file, found in configs->System.

```
{
    "load_m": 0.6066,
    "load_b": 0.9478124999999462,
    "moment_of_inertia": 0.1058,
    "rolling_a": 0.158166318,
    "rolling_b": -116.12,
    "rolling_c": 777.6214652
}
```

Reference each section to calibrate that part of the dyno. The dyno must be calibrated in this order: Torque->Moment of Inertia->Rolling Resistance. The torque can be reaclibrated alone, and is simple to do. However, if the moment of inertia calibration is altered the rolling resistance must also be recalibrated. As such, it is recommended by this guide to check and update the torque calibration regularly, but this guide does not provide any guidance or recommendations on when to recalibrate the moment of inertia and rolling resistance adjustments - these need to be determined by a well-educated user.

5.1.2 Torque Calibration

Torque calibration is the simplest calibration procedure, and can be easily confirmed as accurate whenever the user would like.

To confirm the torque calibration is accurate:

1. Ensure the torque output reads zero. If not, utilize the "Zero Torque Reading" button in the User Terminal, found in the Controls page (F4).
2. Place the torque wrench adapter plate on the back side of the water brake. It is highly recommended to use a digital torque wrench that displays the highest torque seen, rather than just the "click" torque setting.
3. Determine the torque values desired to be tested. It is recommended to test: 25, 50, and 75 ft-lbs.
4. In Grafana or influxdb, view the dynoLoad field. Reach each torque setting with the torque wrench on the adapter plate. Confirm the dynoLoad field matches these values (expect variance of 1 ft-lbs).

If the torque setting maintains proper calibration, no further action is needed. If not, proceed using the steps below:

1. Set constants for slope-intercept form (y=mx+b) in the dyno_mechanical.json file. Set load_m to 1, and load_b to 0. Restart the main.py program if it is running to load these values.

2. Place the torque wrench adapter plate on the back side of the water brake. It is highly recommended to use a digital torque wrench that displays the highest torque seen, rather than just the "click" torque setting.

3. Determine the torque values desired to be tested. It is recommended to test: 0, 25, 50, and 75 ft-lbs.

4. In Grafana or influxdb, view the dynoLoad field. Reach each torque setting with the torque wrench on the adapter plate. Tabulate the results, with "x" being the value of dynoLoad at each torque value, and "y" being the actual torque reached with the torque wrench.

5. Create a line of best fit with this data. This can be done simply in Excel with a linear trendline. The y-intercept should be entered into load_b, and the slope should be entered into load_m in the json file. Restart the main.py program if it is running to load these values.

6. The torque setting is now calibrated. It is recommended to use the above steps to verify the calibration before proceeding.

5.1.3 Moment of Inertia Calibration

The moment of inertia can be calibrated one of two ways. Both of these strategies assume the engine makes the same torque at the same engine speed during multiple differing ocasions. As a result, it is recommended to use a realiable and well setup ICE, as some EV powetrains experience thermal throttle or other torque limiters.

Note that both of these methods can or must use data downloaded from influxdb. Unfortunately, the indfluxdb csv format is not conducive to plotting or comparing data. This can be rectified with an Excel pivot table, or an example of a MatLab script that performs the transformation can be seen in Appendix A.

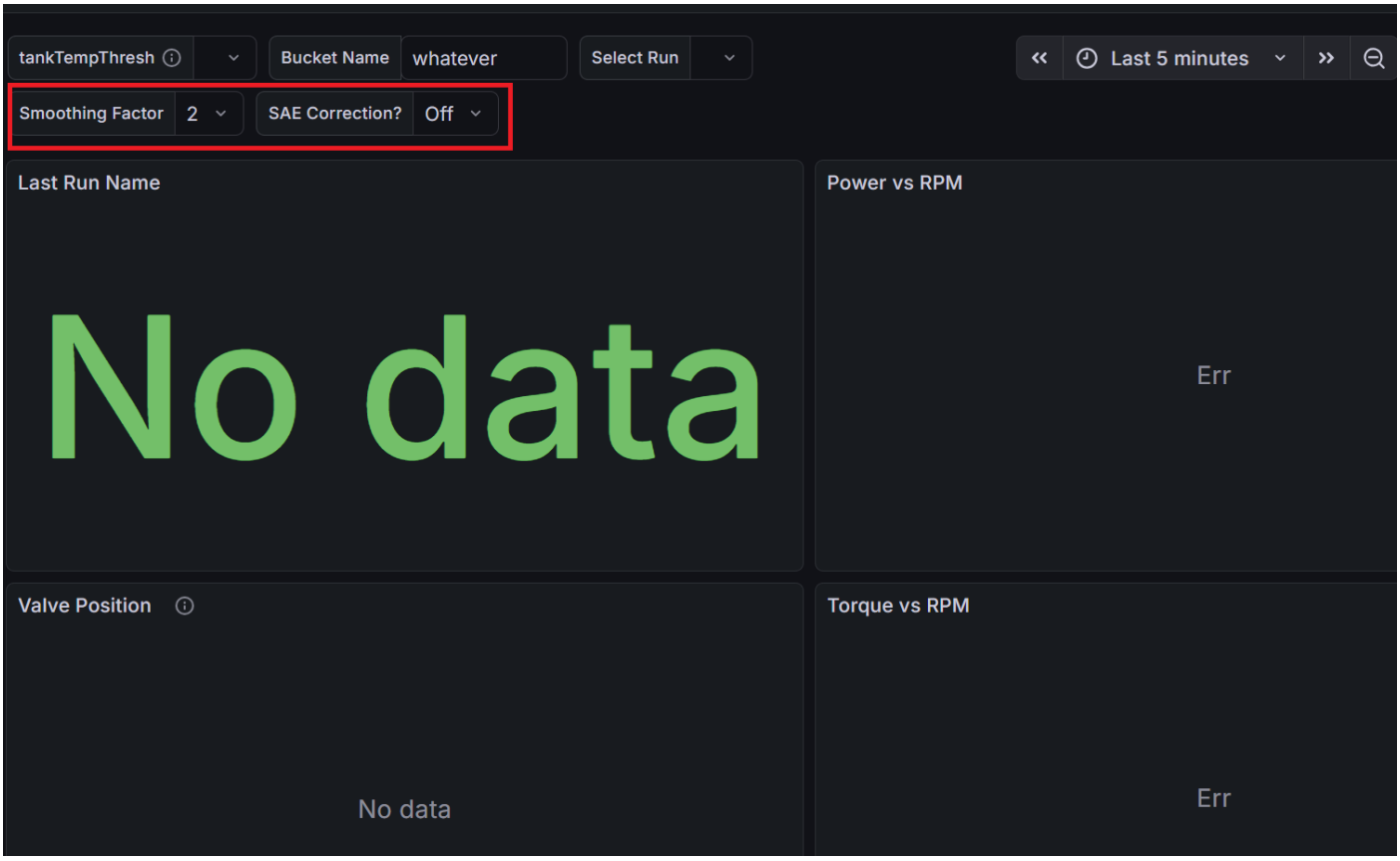
1. Measure the steady-state torque produced by the powertrain at one or multiple engine speeds. Then, place no load on the engine and accelerate through these speeds. Use the acceleration and torque to find the moment of inertia using Newton's 2nd law.

2. Set up multiple pulls with differing ramp rates. Use the differing ramp rates and torques to solve for the moment of inertia, using Newton's 2nd law as the governing equation (assume the powertrain makes equal torque during multiple runs for each speed value). This method is executed in a MatLab script that can be seen in Appendix B.

6 Guidance on Interpreting Data, Engine Types, and Configurations

6.1 Data in Grafana

There are currently two drop-down selections that can be made that affect the data displayed in Grafana. These are the smoothing factor, and SAE correction factor:



6.1.1 Smoothing Factor

The smoothing factor can be selected from a list of integers: 0, 1, or 2. These are arbitrary numbers that roughly correlate to the time constant in an IIR filter applied to the torque value. Selecting a higher smoothing factor makes the data more smooth, intuitively. However, some data may be lost or attenuated if the torque value changes very quickly. It is recommended to use the minimum smoothing factor that makes the data readable and presentable.

6.1.2 SAE Correction Factor

The SAE correction factor can be set to "Off" or "On". This correction factor uses the SAE J1349 standard to adjust power and torque readings for a standard environmental condition (pressure, temperature, humidity) using the currently measured pressure, temperature, and humidity. In Golden, CO this factor is typically between 1.2-1.25 (ie. torque values are multiplied by 1.2-1.25 to match the higher engine output seen at sea level). To emphasize, this value is calculate using the actual measured values at the time and place of operation, so it can be trusted to account for various locations and weather conditions.

Importantly, the SAE correction factor does not apply to all engines. It can be used with the highest confidence on naturally aspirated internal combustion engines. See the engine types section on discussion of when the SAE correction factor is applicable.

6.2 Engine Types

6.2.1 Internal Combustion - Naturally Aspirated

The naturally aspirated internal combustion engine is the least nuanced engine type to test on the water brake dynamometer. Due to the decreased torque output, particularly at low engine speeds, the water brake does not have difficulty braking and measuring torque. Additionally, the torque curve is smooth and repeatable, which results in easy to read torque curves.

The SAE correction factor, as discussed in 6.1.2, can be used to adjust the measured torque and power for standard environmental conditions.

6.2.2 Internal Combustion - Forced Induction

Forced induction engines - turbocharged or supercharged - generally produce more torque and power than similarly sized naturally aspirated engines. Importantly, forced induction engines typically feature a boost controller that targets specified intake manifold pressures. The implication of this is that the SAE correction factor *should not be used for forced induction engines.* The forced induction system typically attenuates the effects of low atmospheric pressure at high altitude, so the correction factor will present higher values than what are correct. If the user desires to adjust the collected data, calculations must be made manually based off the actual intake manifold pressure and the expected intake manifold pressure at sea level. This is beyond the scope of this guide, and it is recommended to interpret the power and torque readings as-is.

Supercharged engines produce torque curves that are similarly shaped to NA engines, but elevated due to the higher effective displacement of the engine. This does not present a problem for the water brake dyno provided that the power output of the engine is within operating specifications, and the gear ratio is correctly selected. See 6.3 on configuration settings.

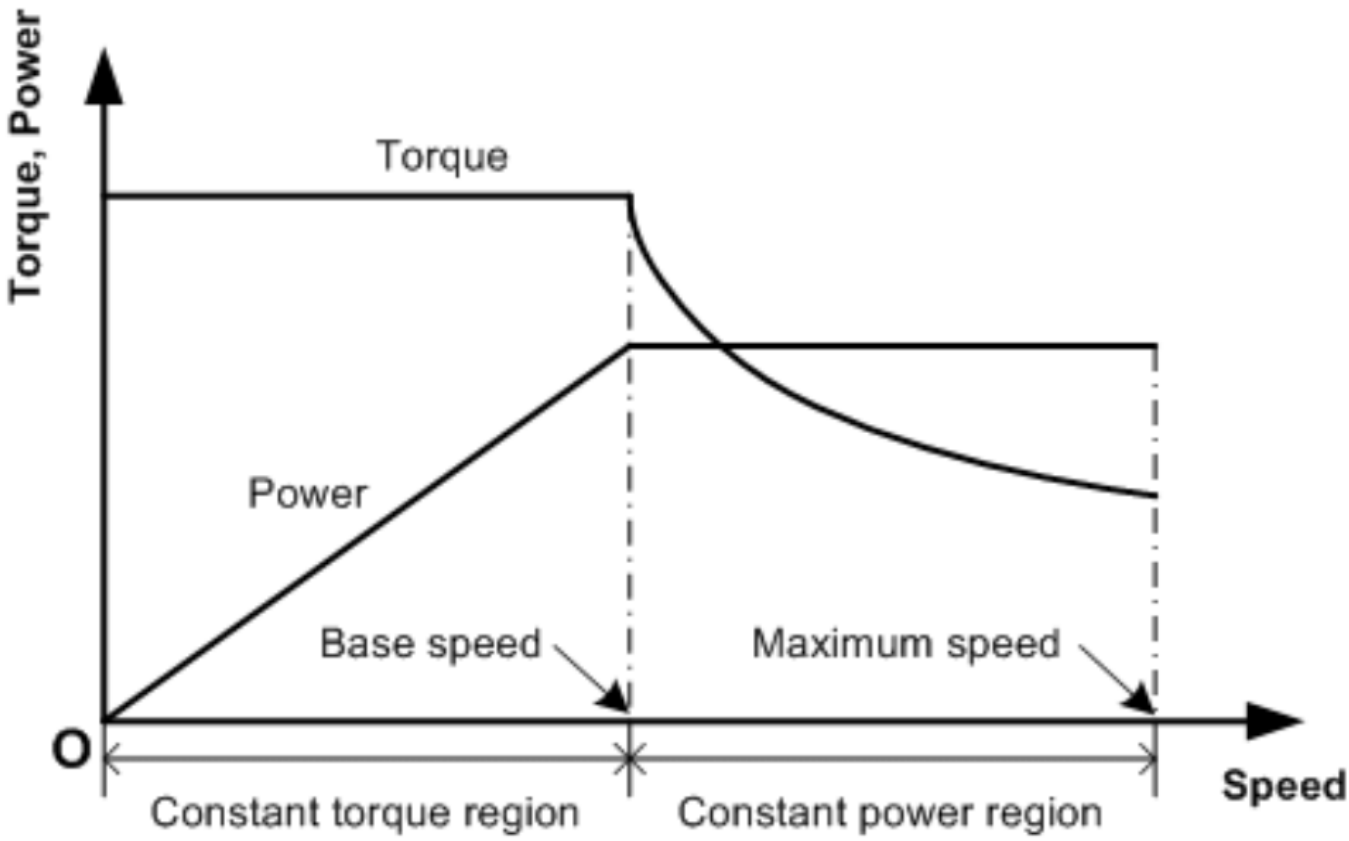
Turbocharged engines, however, have uniquely shaped torque curves. The torque of a turbocharged engine typically rises very quickly at the boost point. There are two scenarios to keep in mind: the boost point is above the target start RPM, and the boost point is below (before) the target start RPM.

In the first case (boost comes in after the pull is started), the dyno should not have great difficulty in collecting and presenting data. The torque and power will likely be compensated for using the moment of inertia of the dyno system if the sudden torque surge overpowers the water brake dyno. The accuracy of this measurement is less than that of steady-state conditions - the dynamometer is not designed as an inertial dyno, and as such inertial calibration should not be held to the same standard as steady-state readings.

In the case where the boost point is below or at the start RPM, the user must check that the engine does not produce more torque than the dyno can brake at this RPM. This can simply be done by loading a configuration file and/or run plan with this target RPM, applying max throttle, and observing whether the dyno can stabilize at the target RPM. If the water brake cannot hold the target RPM low enough, reference section 6.3.3 - High Torque for setup and procedures for measuring power and torque. In the case the dyno can hold the target RPM, proceed as normal with confidence in the measurements.

6.2.3 EV Powertrains

EV powertrains feature unique torque curves, where there is a constant torque reagion and a constant power region. Below is an ideal example of such regions:



[12]

As a gross simplification, the EV motors expected to be tested on this water brake dynamometer will feature lower constant-power output levels than internal combustion engines. As such, this constant power region is typically trivial to measure, and can be treated similarly to an NA internal combustion engine.

The constant torque region, however, can be troublesome for this water brake dyno. The user should determine the minimum RPM they want tested by the dyno, and must check that the engine does not produce more torque than the dyno can brake at this RPM. This can simply be done by loading a configuration file and/or run plan with this target RPM, applying max throttle, and observing whether the dyno can stabilize at the target RPM. If the water brake cannot hold the target RPM low enough, reference section 6.3.3 - High Torque for setup and procedures for measuring power and torque. In the case the dyno can hold the target RPM, proceed as normal with confidence in the measurements.

Importantly, the SAE correction as outlined in 6.1.2 is not applicable to EV motors and must be set to "Off" when interpreting data collected.

Finally, some EV powertrains feature BMS's (battery management systems) that may modulate power and torque output using its own control scheme. This can result in oscillitory behavoir in the measured torque and power, which can make it difficult to compare measurements side by side. It is recommended to incrementally slow the ramp rate when this is the case.

.

6.3 Configurations

The dyno must be configured in multiple ways by the user: physical configuration consists of the setup mode (engine stand dyno or chassis dyno) and the gear ratio selected while doing so. Run plan

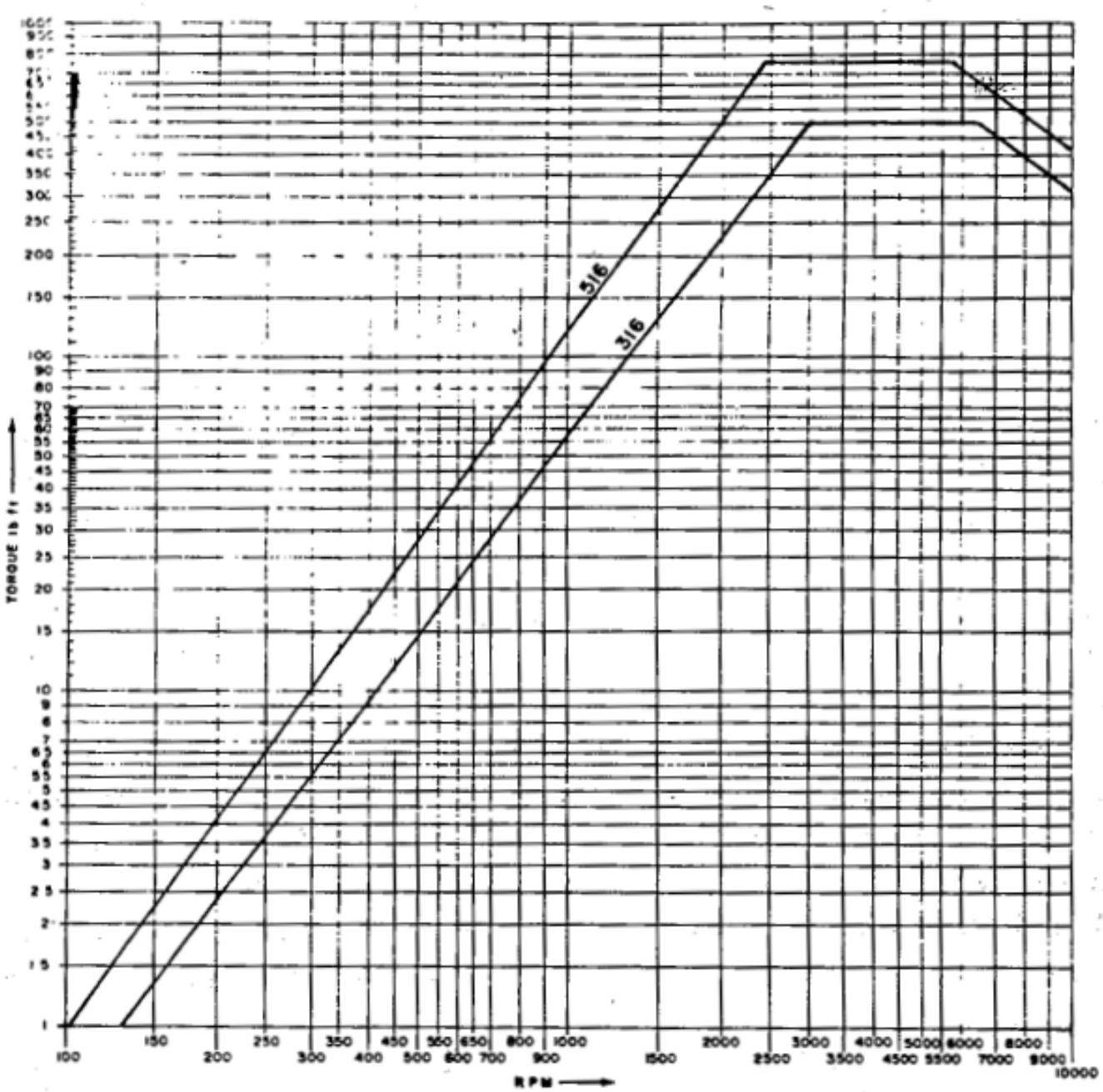
[12]Design and Fem Analysis of Low Voltage Outer Rotor Line Start Permanent Magnet Synchronous Motors with Different Magnet Alignments - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Power-and-torque-demand-curve-in-electric-vehicles_fig2_320611871 [accessed 8 Apr 2026]

configurations determine how the dyno applies load to the engine, and as a result determines the data presented to the user.

6.3.1 Physical

The dyno can be placed in two primary configurations for measuring engine output: the engine stand configuration and the chassis dyno configuration.

The engine stand configuration consists of a direct chain drive from the output sprocket of an engine to the input sprocket (connected via extension shaft). The sprocket sizing determines the gear ratio from the engine to the water brake turbine, which directly affects the speed and torque values seen at the turbine input shaft.



[13] The figure above shows the torque absorbing capacity of the DA-316 water brake turbine (ensure the user is referencing the 316 curve). The rpm denotes the turbine speed - when using the engine stand this is equivalent to the rotational speed of the sprocket. The user should choose a gear ratio that allows the water brake to absorb the torque produced by the engine (remember adjust both torque and speed at the turbine shaft for the selected gear ratio), if possible.

[13] https://gopowersystems.com/wp-content/uploads/2021/06/D_316_516_Manual.pdf

Maximum Turbine Speed

The operation procedures of this dyno limits the maximum input speed of the turbine with the extension shaft connected to 3800 RPM. If a gear ratio does not exist that allows the water brake to absorb the torque while also staying under 3800 RPM, consider a different engine or use the high torque mode outlined in 6.3.3. The turbine speed is accessible via the influxdb field "turbineSpeed"

In the chassis dyno configuration, the minimum road speed at which 100Hp can be absorbed by the dyno without high torque mode is 48mph. However, it may be simpler on vehicles fitted with gearboxes to experimentally determine the minimum gear required before the dyno can brake the engine speed desired. If the dyno does not have enough braking capacity, incrementally increase the gear until it does. If it still does not hold rpm in high gear, reference the high torque section (6.3.3).

6.3.2 Controls Configurations (Run Plans)

There are two primary modes for the dynamometer - hold mode and ramp mode. Before configuring either of these, ensure the gear ratio is calculated as outlined in 4.7.2 Gear Ratio Calculation.

Hold mode is trivial, where it sets a target RPM value and will modulate the load on the powertrain to maintain this engine speed. In hold mode, the user can confirm the dyno is able to brake the power delivered from the motor by observing that the engine RPMs maintain the targeted engine RPMs and that the valve position is not clamped to fully open (100%).

Ramp modes are more nuanced, and it takes some trial and error to obtain the desired data. Below outlines general steps that should be taken by the user when setting up ramp run plans:

1. Determine start and end engine speeds for the ramp ie. the range of engine speeds the user desires to collect data on

2. Check that the water brake can absorb the torque generated at the start engine speed. This can be done either with a hold RPM plan or creating a normal ramp plan, which automatically targets the start RPM when loaded. See 6.3.3 high torque section for more information. If the high torque scenario applies as in the aforementioned section, add the "HiTorque" flag to the run plan.

3. Set a starting rpm ramp value. It is recommended to start with 10 second pulls, and the rate can be calculated as: $rate = \frac{RPM_{end} - RPM_{start}}{Pull\ Duration\ (s)}$

4. Perform a test pull. Iterate through the process by changing parameters and performing test pulls to check that the output data improves. General guidance is as follows: slowing down pull rates, up to a reasonable pull duration for the powertrain, generally improves most problems. When the data at the start RPM value is not clean, it can help to lower the start RPM below the actual value the user is concerned with.

5. After the user has pinned down appropriate start, end, and ramp rate values, it can be deliberated whether the smooth start mode is desired. See 6.3.4 Smooth Start for more information and how to enable the flag. This mode limits time under load by starting a pull through the start RPM rather than waiting for the engine speed to stabilize, although it requires additional configuration whenever the start RPM is changed. Additionally, it leans into the inertial calibration of the dynamometer which does not present the same accuracy as the heavy braking runs. The initial load value is found using the procedure in Step 2 above, and writing down the minimum valve opening that holds the engine at the start RPM in steady-state conditions.

6.3.3 High Torque

The high torque scenario can happen with EV powertrains, forced induction engines, or powertrains where the ideal gear ratio is not possible for measuring steady-state torque and power, particularly at low RPMs.

The high torque scenario applies when the user uses the following procedure to test: The user should determine the minimum RPM they want tested by the dyno, and must check that the engine does not produce more torque than the dyno can brake at this RPM. This can simply be done by loading a configuration file and/or run plan with this target RPM, applying max throttle, and observing whether the dyno can stabilize at the target RPM. If the water brake cannot hold the target RPM low enough, the high torque scenario applies.

In the case the dyno can hold the target RPM, the high torque scenario does not apply and the user should proceed as normal. If the high torque scenario is used when it does not apply, the powertrain will not be able to reach the start RPM and a run will not be possible.

Below is an example json file showing how the high torque condition must be communicated to the dynamometer.

```
{
    "Mode": "Ramp",
    "Start": "3000",
    "End": "6000",
    "Rate": "500",
    "HiTorque": "On"
}
```

The system responds to a high torque scenario as follows: The user presses "Enter" in the user terminal to start a run. The valve position is set to full open, meaning the water brake will be applying as much braking torque as possible. The user then begins a pull by actuating the throttle. Because it has been previously determined the output torque of the engine exceeds the braking torque of the dyno, the motor speed will reach and exceed the start RPM even with full load applied. Once the engine speed does reach the start RPM, the PID control will be enabled "on the fly" and a ramp will be carried out as normal. It should be expected that the engine RPM remains above the target RPM for a portion of the pull. The pull is ended as normal (bell rings, user lets off throttle).

Notably, the high torque scenario relies largely on the moment of inertia of the dyno system to calculate torque and power in the lower RPM region. The accuracy of this measurement is less than that of steady-state conditions - the dynamometer is not designed as an inertial dyno, and as such inertial calibration should not be held to the same standard as steady-state readings.

This strategy, however, does allow the user to test powertrains that would otherwise be impossible on small water-brake dynamometers, as this is an inherent weakness of the dyno style.

6.3.4 Smooth Start

The smooth start configuration is designed for when the user wants to make many pulls with the same configuration (powertrain, gearing, run plan) and would like to limit the amount of time the powertrain dwells at low speeds while waiting for the water brake to stabilize. It works similarly to the high torque scenario (see 6.3.3), in that it hard codes a valve position until the ramp is started automatically from the user exceeding the start RPM. Unlike the high torque scenario, the valve position is not set to fully open, and must be configured by the user using the steady-state valve position found when holding the engine at the start RPMs.

```
{
    "Mode": "Ramp",
    "Start": "3000",
    "End": "6000",
    "Rate": "500",
    "SmoothStart": "On",
    "InitialLoad": 70
}
```

To enable smooth start, the "SmoothStart" field in the run plan json is set to "On", and the "InitialLoad" field is set to the observed steady-state valve position. The program automatically reduces this value by 10% to ensure the engine has enough power to reach the start RPMs.

As mentioned, the smooth start feature is designed when users would not like to hold the load on the engine in the low range for extended periods while waiting for the dyno controls to stabilize. However, if the user chooses to change the start RPM the initial load must be recalibrated. In the case the user would like to experiment with start RPMs, it is recommended to use the standard run plan, and transition to the smooth start once they are confident they have found the range they are interested in.

If the user finds the engine RPMs rise too quickly or too slowly up to the start RPM, the initial load can manually be adjusted to be higher or lower as a correction, respectively.

7 Appendix

A Data Processing/Transformation Script

```
%%
data = readtable("MF13_4-4-26.csv");


runNames = unique(data.runName);

runs = {};
for i = 1:length(runNames)
    name = runNames{i};
    withinRun = data(strcmp(data.runName, name), :);
    time = unique(withinRun.time);
    rawSpeed = zeros(length(time), 1);
    wheelSpeed = zeros(length(time), 1);
    dynoLoad = zeros(length(time), 1);
    for t = 1:length(time)
        atTime = withinRun(strcmp(withinRun.time, time(t)), :);
        rawSpeed(t) = atTime.value(strcmp(atTime.measurement, ...
        "rawSpeed"));
        wheelSpeed(t) = atTime.value(strcmp(atTime.measurement, ...
        "wheelSpeed"));
        dynoLoad(t) = atTime.value(strcmp(atTime.measurement, ...
        "dynoLoad"));
    end
    time = strip(extractAfter(time, 12), "right", "Z");
    run = sortrows(table(time, dynoLoad, wheelSpeed, rawSpeed));
```

```
    fileName = name + "_4_4_26";
    writetable(run, fileName);
end
```

B Moment of Inertia Script

```
%%
run1 = readtable("MF13_6th_2_4_4_26.txt");
run2 = readtable("MF13_6th_9_4_4_26.txt");
run3 = readtable("MF13_6th_3_4_4_26.txt");

Fs = 20; %sampling rate
GR = 4.304; %gear ratio for pulls

w_1 = run1.wheelSpeed;
t_1 = run1.dynoLoad;

w_2 = run2.wheelSpeed;
t_2 = run2.dynoLoad;

w_3 = run3.wheelSpeed;
t_3 = run3.dynoLoad;

%delete duplicate speed values
[w_1_unique, id_w1] = unique(w_1);
t_1_unique = t_1(id_w1);

[w_2_unique, id_w2] = unique(w_2);
t_2_unique = t_2(id_w2);

[w_3_unique, id_w3] = unique(w_3);
t_3_unique = t_3(id_w3);

%window and order for Savitzky-Golay filtering
window = 91;
order = 1;

%apply filter for each value to smooth data out.
%This is a different filter than used in realtime, but produces great
%results.
w_1 = sgolayfilt(w_1_unique, order, (window*4 - 1));
t_1 = sgolayfilt(t_1_unique, order, 11) * 5/4;
a_1 = Fs*gradient(w_1); %get acceleration as numerical derivative

w_2 = sgolayfilt(w_2_unique, order, (window*2 - 1));
t_2 = sgolayfilt(t_2_unique, order, 11) * 5/4;
a_2 = Fs*gradient(w_2);

w_3 = sgolayfilt(w_3_unique, order, window);
t_3 = sgolayfilt(t_3_unique, order, 11) * 5/4;
a_3 = Fs*gradient(w_3);
```

```
%%
% yyaxis right
% plot(w_1, a_1)
% hold on
% plot(w_2, a_2)
% plot(w_3, a_3)
% yyaxis left
% plot(w_1, t_1)
%
% plot(w_2, t_2)
% plot(w_3, t_3)
% hold off
%%
%%rearranging to t and a vs w
t1vw = @(w) interp1(w_1, t_1, w);
t2vw = @(w) interp1(w_2, t_2, w);
t3vw = @(w) interp1(w_3, t_3, w);

a1vw = @(w) interp1(w_1, a_1, w);
a2vw = @(w) interp1(w_2, a_2, w);
a3vw = @(w) interp1(w_3, a_3, w);

%%Find I values
W = linspace(w_1(1), w_1(end), 1000);
%%
%compare each run to another to solve for moment of inertia. Some runs
%compare well (produce finite, real results) where others do not (when
%accelerations match up too closely).
%This gives each moment of inertia as a function of roller speed
I_12 = (t2vw(W)-t1vw(W))./(a1vw(W)-a2vw(W));
I_13 = (t3vw(W)-t1vw(W))./(a1vw(W)-a3vw(W));
I_23 = (t3vw(W)-t2vw(W))./(a2vw(W)-a3vw(W));


plot(W, I_13)
hold on
plot(W, I_12)
plot(W, I_23)
title("Calculated I vs RPM")
legend()
%calculate moment of inertia as average of all moment of inertias
I_av = mean([mean(I_13, 'omitnan'), mean(I_23, 'omitnan'), ...
mean(I_12, 'omitnan')])
I_av = mean([mean(I_13, 'omitnan'), mean(I_12, 'omitnan')])
ylim([-10 10])
hold off
%%
```